P P SAVANI UNIVERSITY

SCHOOL OF ENGINEERING

INTERNSHIP / UDP / TRAINING

FINAL PROJECT REPORT

(Hard-Bound Project Report Format)

Particular	Details
Project Title	Cache-Craft: Autonomous Semantic Cache Router - A High-Performance Multi-Tier Semantic Caching Middleware for LLM Inference
Student Name	Aakash Bharatbhai Jayani
Enrollment Number	23SE02IT140
Program / Branch	B.Tech / Information Technology
Semester	7th Semester
Company / Organization	P P Savani University (University Development Project / UDP)
Department / Domain	Information Technology — Semantic Caching and LLM Middleware
Mentor (Company)	Not Applicable
Mentor (Institute)	Mr. Anurag Yadav
Academic Year	2026 - 2027

CERTIFICATE

OF COMPLETION

This is to certify that

AAKASH BHARATBHAI JAYANI

Enrollment No.: 23SE02IT140

of B.Tech / Information Technology, 7th Semester, has successfully completed the University Development Project (UDP) titled

"Cache-Craft: Autonomous Semantic Cache Router"

during the period from 08 June 2026 to 03 October 2026 at P P Savani University, Surat, under the guidance of Mr. Anurag Yadav.

The project work, final implementation, testing and validation, documentation, and demonstration were completed as required for the Internship / UDP / Training and found satisfactory.

Date of Issue: ___ / ___ / _____

_____	_____
Mentor (Company)	**Mentor (Institute)**
(Not Applicable)	Mr. Anurag Yadav

DECLARATION

I hereby declare that this report is a consolidated and authentic record of the UDP project work carried out by me. All sources used have been appropriately acknowledged and cited.

Place: _____

Date: ____ / ____ / _____

Signature of Student: _____

Aakash Bharatbhai Jayani (Enrollment No.: 23SE02IT140)

ACKNOWLEDGEMENT

I take immense pleasure in expressing my profound gratitude and indebtedness to P P SAVANI UNIVERSITY, SCHOOL OF ENGINEERING, and the Department of Information Technology for providing an intellectually stimulating environment and outstanding computational facilities that made the successful execution of this University Development Project possible.

I express my deepest gratitude to my esteemed Institute Guide and Mentor, Mr. Anurag Yadav, for his invaluable guidance, keen technical insights, constant encouragement, and constructive critique throughout the developmental lifecycle of Cache-Craft. His deep knowledge of distributed systems and machine learning pipelines proved instrumental in shaping the multi-tier routing architecture, resolving complex vector distance space anomalies, and establishing rigorous empirical benchmarking methodologies.

I also extend my sincere appreciation to the Head of the Department and the faculty members of the Department of Information Technology for their continuous academic support, constructive evaluations during the four milestone reporting defenses, and helpful feedback on system design and validation.

Special thanks are due to my peers and fellow researchers who participated in user acceptance testing and provided insightful feedback on the interactive chat playground and real-time observability telemetry. Lastly, I am eternally grateful to my parents and family for their unwavering moral support, patience, and encouragement throughout my academic journey.

Aakash Bharatbhai Jayani

23SE02IT140

ABSTRACT / EXECUTIVE SUMMARY

The rapid industrial proliferation of Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) pipelines has created a critical operational bottleneck characterized by excessive inference latency (often exceeding 2,000 to 4,000 milliseconds) and steep recurring financial expenditures driven by per-token cloud API billing. While traditional web architectures rely heavily on deterministic key-value caching, conventional exact-match caches fail entirely in generative AI workflows because natural language queries expressing identical semantic intent exhibit infinite syntactic variations. To overcome this fundamental limitation, this project designs, implements, and empirically evaluates Cache-Craft: an autonomous, multi-tier semantic cache router and intelligent caching middleware.

Cache-Craft uses a three-tier cache path: (1) L0, a process-local in-memory exact-query cache keyed by a normalized SHA-256 hash; (2) L1, a Redis exact-hash cache; and (3) L2, a persistent semantic vector cache in PostgreSQL 16 with pgvector and an HNSW index. Incoming queries are normalized and embedded with sentence-transformers/all-MiniLM-L6-v2 to produce 384-dimensional vectors. Cosine similarity is compared with a configurable threshold. The reported benchmark identifies $\tau = 0.70$ as the F1-optimal threshold for its evaluated corpus; runtime settings may use a more conservative threshold. When a query does not match the cache, the middleware calls the configured inference provider. The codebase supports Google Gemini through its configured model IDs and local Ollama inference in an environment where that service is reachable and authorized. The hosted backend cannot directly access the developer workstation GPU, so local and cloud execution must be described as distinct deployment contexts.

The benchmark corpus contains 1,200 items: 500 seed queries used to populate the cache, 500 paraphrases, and 200 unrelated negative controls. The reported threshold evaluation therefore uses 700 evaluated queries after seeding. At $\tau = 0.70$, the recorded matrix reports an F1-score of 0.9806, precision of 1.0000, and recall of 0.9620. Across those 700 evaluated requests, 481 cache hits correspond to a 68.71% hit rate. Using the stated mean latencies of 2,400.0 ms for the uncached baseline and 768.26 ms for the mixed workload yields a ratio of approximately 3.12× and an average latency reduction of about 68.0%. The reported 68.71% API-cost reduction is a benchmark-model estimate based on the assumption that cache hits avoid the

corresponding upstream generation cost. The project includes a Next.js dashboard with Server-Sent Events (SSE) telemetry, prompt-history navigation and device-authorization controls; the precise operating boundaries and evidence are described in the following chapters.

Keywords: Semantic Caching, Large Language Models (LLMs), Retrieval-Augmented Generation (RAG) Compatibility, pgvector, Hierarchical Navigable Small World (HNSW), Cosine Similarity, FastAPI, Next.js 16, Inference Routing, Benchmark Cost Estimate, Ollama, Google Gemini.

TABLE OF CONTENTS

Chapter / Section	Title	Page No.
	Preliminary Pages	
	Certificate	i
	Declaration	ii
	Acknowledgement	iii
	Abstract / Executive Summary	iv
	Table of Contents	vi
	List of Figures	x
	List of Tables	xi
	List of Abbreviations / Acronyms	xiii
Chapter 1	**Introduction**	1
1.1	Background of the Project	1
1.2	Problem Statement	2
1.3	Need / Motivation for the Project	2
1.4	Objectives of the Project	3
1.5	Scope of the Project	3
1.6	Expected Outcomes vs. Final Achieved Outcomes	4
1.7	Report Organization	5
Chapter 2	**Organization / UDP Profile**	7
2.1	Organization / Institutional Overview	7
2.2	Department / Domain Profile	7
2.3	Role and Individual Responsibilities	8
2.4	Development / Work Environment & Specifications	8
2.5	UDP Project Administrative Details	9
Chapter 3	**Literature Review / Background Study**	10
3.1	Domain Background	10
3.2	Existing Approaches & Related Work	10
3.3	Comparative Analysis	11
3.4	Research Gap and Project Justification	12
Chapter 4	**Requirements Analysis and Project Scope**	14
4.1	Functional Requirements Specification	14
4.2	Non-Functional Requirements Specification	15
4.3	Hardware Requirements and Infrastructure	16
4.4	Software Requirements and Dependencies	16
4.5	Data / Dataset Requirements	17
4.6	System Constraints and Operational Assumptions	18
Chapter 5	**Methodology and Project Planning**	19
5.1	Final Development and Research Methodology	19

Chapter / Section	Title	Page No.
5.2	Project Workflow & Process Architecture	19
5.3	Project Phases and Deliverables	20
5.4	Progress Report Consolidation	21
5.5	Project Schedule and 16-Week Gantt Timeline	21
Chapter 6	**System Analysis and Design**	23
6.1	Existing System / Current Practice	23
6.2	Proposed System	23
6.3	System Architecture	24
6.4	Workflow / Process Flow	25
6.5	UML / Design Diagrams	25
6.6	Module-wise Detailed Design	27
6.7	Database / Relational pgvector Data Design	29
6.8	User Interface & Experience Design System	30
Chapter 7	**Technology Stack and Development Environment**	31
7.1	Programming Languages	31
7.2	Frameworks, Libraries & External APIs	31
7.3	Database & In-Memory Storage Systems	32
7.4	Hardware, Cloud & Deployment Environments	32
7.5	Development, Version Control & Testing Tooling	32
7.6	Technology Selection Rationale	33
Chapter 8	**System Implementation**	34
8.1	Implementation Architecture Overview	34
8.2	Module 1: Multi-Tier Cache Router Core	34
8.3	Module 2: Dense Vector Embedding Engine	34
8.4	Module 3: Dynamic Multi-LLM Router & Failover Engine	34
8.5	Module 4: Hardware Device Authorization & Isolation Boundary	35
8.6	Module 5: Real-Time Observability & SSE Telemetry Engine	35
8.7	Module 6: Interactive Web Dashboard & Chat Playground	35
8.8	Module 7: Automated Benchmarking & Matrix Validation Suite	35
8.9	Mathematical Formulations: Cosine Metric & HNSW Graph Indexing	35
8.10	RESTful API Endpoints Specification	36
8.11	Database Implementation & pgvector DDL Excerpts	37
8.12	Security Architecture, CORS & GPU Access Control	37

Chapter / Section	Title	Page No.
Chapter 9	**Testing and Validation**	39
9.1	Multi-Tiered Testing Strategy & Verification Methodology	39
9.2	Documented Unit and Module Scenarios (25)	39
9.3	Documented Functional Scenarios (40)	40
9.4	Documented Integration Scenarios (35)	41
9.5	Documented System and Resilience Scenarios (25)	42
9.6	Documented Performance, Load and Concurrency Scenarios (20)	43
9.7	Documented User Acceptance Testing (UAT) Scenarios (18)	44
9.8	Experimental Validation on 1,200 Benchmark Corpus	45
9.9	Defects Discovered, Root Causes & Technical Resolutions	45
9.10	Documented Verification Matrix and Validation Summary	46
Chapter 10	**Results and Performance Analysis**	47
10.1	Final Empirical Results Summary	47
10.2	Objective-wise Milestone Achievement	47
10.3	Similarity Threshold Sweep & Metric Analysis	48
10.4	Baseline Comparative Results	48
10.5	Empirical Charts & Production Screenshots	49
10.6	Results Discussion	52
Chapter 11	**Challenges, Solutions and Improvements**	53
11.1	Major Architectural & Algorithmic Challenges	53
11.2	Technical Challenges & Engineering Resolutions	53
11.3	Process, Dataset & Environment Challenges	54
11.4	Architectural Improvements Made During Development	54
11.5	Lessons Learned from Systems Engineering Challenges	55
Chapter 12	**Individual Contribution and Professional Learning**	56
12.1	Individual Contribution & Project Ownership	56
12.2	Technical Competencies & Skills Acquired	56
12.3	Professional, Research & Architectural Skills Acquired	56
12.4	Major Academic and Professional Learning Outcomes	57
Chapter 13	**Limitations and Future Scope**	58

Chapter / Section	Title	Page No.
13.1	Current Technical Limitations	58
13.2	Future Scope & Developmental Roadmap	58
13.3	Scalability, Clustering & Enterprise Extension Opportunities	59
Chapter 14	**Conclusion**	60
14.1	Final Project Completion Status	60
14.2	Concluding Remarks	61
Chapter 15	**References**	62
15.1	References	62
Chapter A	**Appendices**	65
Appendix A	Progress Report 1 Summary (Foundations & Concept)	65
Appendix B	Progress Report 2 Summary (Prototype & Mathematical Proof)	65
Appendix C	Progress Report 3 Summary (pgvector & Frontend V1)	65
Appendix D	Reporting 4 / Final Evaluation Summary (Multi-Tier & Benchmarking)	66
Appendix E	Additional Benchmark Visualizations & Telemetry Callouts	66
Appendix F	Certificates & Academic Evidence	66
Appendix G	Source Code, Git Repository & Cloud Deployment Evidence	67
Appendix H	Mentor / Evaluator Remarks	67

LIST OF FIGURES

Figure 5.1: Cache-Craft End-to-End System Development and Research Workflow 20
Figure 6.1: Runtime Architecture - L0 In-Process Cache, L1 Redis, L2 pgvector and Inference Providers 24
Figure 6.2: Request Routing Flow Through L0, L1, L2 and the Configured Inference Path 25
Figure 6.3: UML and Behavioral Design Overview — Use Case, Conceptual Class, Sequence and Activity Diagrams 26
Figure 6.4: Cache-Craft Component and Deployment-Boundary Diagram 27
Figure 8.1: PostgreSQL pgvector Schema and HNSW Index Initialization (DDL Excerpt) 37
Figure 10.1: Mean End-to-End Latency Comparison for the Post-Seeding Evaluation Workload 49
Figure 10.2: Threshold Sensitivity — Precision, Recall, F1, Hit Rate and Mean Latency 50
Figure 10.3: Next.js 16 Web Dashboard and Real-Time Telemetry Overview 50
Figure 10.4: Interactive Semantic Chat Playground with L1 Cache-Hit Highlighting 51
Figure 10.5: Vector Cache Explorer and Semantic Inverted Index Browser 51
Figure 10.6: Dynamic Similarity Threshold Controls and Device Authorization Panel 52

LIST OF TABLES

Table 1.1: Cache-Craft Project Scope Matrix 4
Table 1.2: Expected vs. Final Achieved Project Outcomes 4
Table 2.1: Engineering Workstation Hardware and System Specifications 8
Table 3.1: Overview of Existing Semantic Caching Approaches 10
Table 3.2: Comprehensive Comparative Matrix (Cache-Craft vs. State of the Art) 11
Table 4.1: Functional Requirements Specification (FR-01 to FR-10) 14
Table 4.2: Non-Functional Requirements Specification (NFR-01 to NFR-08) 15
Table 4.3: Hardware Requirements and Infrastructure Specifications 16
Table 4.4: Software Requirements and Runtime Dependencies 17
Table 4.5: Benchmark Dataset Requirements and Corpus Distribution 18
Table 5.1: Project Phases, Milestones, and Key Deliverables 20
Table 5.2: Progress Report Consolidation and Developmental Evolution 21
Table 5.3: Comprehensive 16-Week Project Schedule and Timeline 21
Table 6.1: Module-wise Architectural Design and Functional Responsibilities 27
Table 6.2: PostgreSQL Database Schema and pgvector Index Definitions 29
Table 7.1: Programming Languages Used 31
Table 7.2: Frameworks, Libraries and External APIs 31
Table 7.3: Database and In-Memory Storage Systems 32
Table 7.4: Hardware, Cloud and Deployment Environments 32
Table 7.5: Development, Version Control and Testing Tooling 32
Table 7.6: Technology Selection Rationale and Architectural Trade-Off Analysis 33
Table 8.1: Cache-Craft RESTful API Endpoints Specification 36
Table 9.1: Unit and Module Verification Scenarios (25) 39
Table 9.2: Functional Verification Scenarios (40) 40
Table 9.3: Integration Verification Scenarios (35) 41
Table 9.4: System and Resilience Verification Scenarios (25) 42
Table 9.5: Performance and Load Verification Scenarios (20) 43
Table 9.6: User Acceptance Verification Scenarios (18) 44
Table 9.7: Major Defects Discovered, Root Causes, and Technical Resolutions 45
Table 10.1: Objective-wise Milestone and Metric Achievement Summary 47
Table 10.2: Empirical Similarity Threshold Sweep and Benchmark Evaluation (N=700) 48
Table 10.3: Baseline Comparative Results (Direct LLM vs. Exact Hash vs. Cache-Craft) 49
Table 11.1: Architectural Improvements and Developmental Refinements 54
Table 12.1: Technical Skills and Engineering Competencies Acquired 56
Table 12.2: Professional, Research, and Project Management Skills Acquired 57

Table 13.1: Current System Limitations and Operational Boundaries 58
Table 13.2: Future Scope, Planned Enhancements, and Scalability Opportunities 59
Table 14.1: Final Comprehensive Project Completion Status Matrix 60
Table A.1: Mentor / Evaluator Remarks Record (for completion by the evaluator) 67

LIST OF ABBREVIATIONS / ACRONYMS

Abbreviation / Acronym	Full Expansion and Engineering Context
RAG	Retrieval-Augmented Generation
LLM	Large Language Model
ANN	Approximate Nearest Neighbor
HNSW	Hierarchical Navigable Small World (Graph Index)
SSE	Server-Sent Events
TTL	Time-To-Live (Cache Expiration Lifecycle)
SHA	Secure Hash Algorithm (specifically SHA-256)
COS	Cosine Similarity / Cosine Distance Metric
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
DDL	Data Definition Language
SQL	Structured Query Language
GPU	Graphics Processing Unit
CPU	Central Processing Unit
RAM	Random Access Memory
OS	Operating System
UAT	User Acceptance Testing
FR	Functional Requirement
NFR	Non-Functional Requirement
UI	User Interface
UX	User Experience
CI/CD	Continuous Integration and Continuous Deployment
MVC	Model-View-Controller Architectural Pattern
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
TCP	Transmission Control Protocol
CORS	Cross-Origin Resource Sharing
CUDA	Compute Unified Device Architecture (NVIDIA)
IVF	Inverted File Index (Vector Quantization)
SLA	Service Level Agreement
UDP	University Development Project (Academic Track)

CHAPTER 1
INTRODUCTION

1.1 Background of the Project

The contemporary computing landscape is experiencing an epochal paradigm shift driven by the rapid maturation and industrial integration of Large Language Models (LLMs) such as OpenAI's GPT-4, Google's Gemini, and Meta's Llama series. Enterprises and academic institutions increasingly deploy these models within Retrieval-Augmented Generation (RAG) pipelines to deliver conversational intelligence, code synthesis, document summarization, and customer support. By anchoring foundation models to external vector knowledge repositories, RAG frameworks dramatically reduce factual hallucinations and empower systems with domain-specific precision.

However, this widespread adoption has exposed severe operational vulnerabilities in modern AI architectures. Foundation model inference is compute-intensive, memory-bound, and economically costly. Cloud providers bill LLM access on a granular per-token consumption model, transforming high-traffic conversational workloads into substantial, recurring operational expenses. Concurrently, inference latency frequently ranges from 1,500 to over 4,000 milliseconds per query, severely degrading the user experience in interactive and real-time environments.

In traditional web systems, high latency and repeated computational costs are systematically mitigated through deterministic key-value caching mechanisms (such as Memcached or standard Redis). Unfortunately, deterministic caching relies on exact cryptographic key matching (e.g., SHA-256 string hashes). In generative AI applications, natural language queries expressing identical underlying user intent exhibit infinite lexical and syntactic variations (e.g., 'What is a semantic cache?' versus 'Can you explain semantic caching?'). Exact-match caches experience a near 100% miss rate against paraphrased inputs, forcing the underlying system to re-invoke expensive LLM inference repeatedly. Cache-Craft was conceived to solve this fundamental architectural bottleneck through high-performance semantic routing and multi-tier vector caching.

1.2 Problem Statement

Modern LLM-powered applications suffer from three interrelated systemic deficiencies that constrain their scalability:

1. **Redundant Cloud API Expenditures:** In conversational RAG systems, up to 60–70% of inbound queries in customer support and documentation search express semantic intent identical to previously resolved prompts. Due to lexical variations, current systems re-execute the entire RAG pipeline, generating millions of redundant tokens and wasting financial capital.
2. **Unacceptable Inference Latency:** End-to-end generation times spanning 2 to 4 seconds introduce cognitive friction, degrade conversational responsiveness, and prevent integration into latency-critical enterprise SLAs.
3. **Single-Point-of-Failure Fragility:** Cloud LLM APIs are inherently susceptible to transient network congestion, rate limiting (HTTP 429), and upstream service outages (HTTP 503). Relying exclusively on single-provider cloud endpoints introduces a brittle single point of failure that halts customer workflows.

Formally, the engineering problem addressed by this project is: How can we architect an autonomous, low-overhead middleware that accurately determines semantic equivalence between incoming natural language prompts and historical responses in real time (<30 ms), serves verified answers from an optimized multi-tier cache, and dynamically orchestrates fallback between cloud and local LLM providers without sacrificing contextual precision or security?

1.3 Need / Motivation for the Project

The motivation for Cache-Craft stems from both academic necessity and pressing industrial demand. As generative AI transitions from experimental prototypes to mission-critical infrastructure, computational efficiency and operational autonomy become paramount. Organizations cannot sustain exponentially growing cloud bills for queries their systems have already answered thousands of times. Furthermore, data privacy mandates require mechanisms that can keep sensitive user queries confined to local hardware without sacrificing performance.

By constructing a multi-tier semantic router that combines sub-millisecond cryptographic hash checks with dense vector similarity searches over an ACID-compliant PostgreSQL 16 pgvector store, Cache-Craft provides an practical bridge

between traditional caching performance and modern generative AI flexibility. Crucially, providing an automated local fallback engine (via Ollama llama3.2) alongside cloud intelligence (Google Gemini) democratizes resilient AI infrastructure.

1.4 Objectives of the Project

The primary engineering and research objectives established for Cache-Craft are:

1. **Multi-Tier Cache Hierarchy:** Implement a request path consisting of L0 process-local in-memory exact lookup, L1 Redis exact-hash caching, and L2 PostgreSQL/pgvector semantic retrieval.
2. **Real-Time Vector Embedding:** Integrate sentence-transformers/all-MiniLM-L6-v2 to produce 384-dimensional dense embeddings; runtime latency depends on the host and no unit-normalization or sub-10ms runtime guarantee is claimed.
3. **Sub-30ms Semantic Similarity Search:** Implement an accelerated Hierarchical Navigable Small World (HNSW) graph index in PostgreSQL pgvector to evaluate cosine distance (<=>) in sub-30 milliseconds.
4. **Rigorous Mathematical Optimization:** Empirically evaluate precision, recall, F1-score, and cost curves across a curated 1,200-query benchmark corpus, identifying the optimal mathematical threshold (tau) to achieve 100% precision.
5. **Fault-Tolerant Multi-LLM Routing:** Architect an automated fallback pipeline that seamlessly redirects traffic from Google Gemini Cloud to a local Ollama llama3.2 instance during API outages or rate limit spikes.
6. **Multi-Turn Context-Aware Caching:** Implement referential intent detection and partition filtering in pgvector to prevent context collision and semantic drift between root queries and conversational follow-ups.
7. **Modern Full-Stack Web Platform:** Build a responsive Next.js 16 dashboard featuring overview, chat, cache and settings views; the overview polls metrics, the backend exposes an SSE endpoint, and local GPU controls require deployment-specific access protection.

1.5 Scope of the Project

The architectural scope of Cache-Craft covers incoming text prompts for LLM applications. The project encompasses embedding generation, multi-tier cache evaluation, vector database indexing, cloud and local LLM routing, empirical

benchmarking and web telemetry. It does not include its own document retriever or RAG context-construction stage. Table 1.1 delineates the project scope.

Table 1.1: Cache-Craft Project Scope Matrix

In Scope (Delivered & Validated)	Out of Scope (Boundaries)	Architectural Remarks
Multi-Tier Cache Routing (L0 in-process cache + L1 Redis exact-hash cache + L2 pgvector)	Cross-modal image/audio embedding caching	L0 provides process-local exact-hash lookup; L1 uses Redis exact-hash caching; L2 handles dense text vectors.
Multi-Tier Cache Routing (L0 process-local dictionary + L1 Redis exact-hash cache + L2 pgvector)	Training or fine-tuning underlying foundation LLM weights	L0 is an unbounded process-local dictionary; L1 uses Redis exact-hash caching; L2 stores semantic vectors.
Hardware Device Authorization & GPU Boundary Protection	Public user login/registration with OAuth2 credentials	Secures local GPU resources from external client exploitation while allowing open API access.
Provider fallback and local inference paths where a configured runtime is reachable	Distributed multi-region active-active cluster failover	Fallback is best-effort; a hosted backend cannot access the developer workstation Ollama/GPU.
SSE endpoint and interactive Next.js 16 UI	Third-party payment gateways or billing subscription engines	Backend SSE emits every two seconds; Overview currently polls every four seconds.
Empirical Multi-Threshold Benchmarking on 1,200 Query Corpus	Benchmarking proprietary closed models beyond Gemini Flash	Rigorous evaluation validates precision, recall, and cost curves across similarity thresholds.

1.6 Expected Outcomes vs. Final Achieved Outcomes

Throughout the four reporting milestones of the UDP lifecycle, project deliverables were tracked against quantitative engineering benchmarks. Table 1.2 presents the comparative evaluation of initial expectations versus final verified outcomes.

Table 1.2: Expected vs. Final Achieved Project Outcomes

Performance Parameter	Initial Expected Target	Final Achieved Outcome (Empirical)
Semantic Similarity Hit Rate	Achieve ≥60% semantic cache hit rate on paraphrased natural language queries.	Achieved 68.71% hit rate on evaluation corpus ($N=700$) at optimal similarity threshold $\tau=0.70$.
Cache Hit Latency Reduction	Serve cached queries in under 50 ms compared to >2,000 ms direct LLM inference.	Achieved 25.33 ms average L2 semantic hit latency and 1.84 ms L0 exact-match path latency (up to 99% reduction).

Performance Parameter	Initial Expected Target	Final Achieved Outcome (Empirical)
Inference Latency Speedup	Achieve overall end-to-end speedup factor ≥2.5× across mixed query workloads.	Computed speedup ratio of approximately 3.12× from the reported mean latencies (2,400 ms / 768.26 ms).
Cloud API Cost Reduction	Reduce external LLM token consumption and API billing costs by ≥50%.	Estimated 68.71% reduction under the benchmark assumption that each cache hit avoids one upstream generation charge; no live token-billing data is recorded.
Semantic Precision SLA	Maintain semantic precision ≥95% to prevent false positives and context hallucinations.	Achieved 100.0% precision (0 false positives) at $τ=0.70$ with F1-score of 0.9806.
Fault Tolerant Fallback	Gracefully handle external cloud API outages (503/429) without crashing client sessions.	Next.js 16 dashboard with Overview, Chat, Cache Explorer and Settings; deployment status is reported, not independently verified here.
Multi-Turn Context Caching	Prevent semantic drift and context collision when caching follow-up questions.	Implemented referential intent detection and pgvector partition isolation for follow-up query turns.
Production Web Dashboard	Deliver modern interactive UI for real-time monitoring and parameter tuning.	Built Next.js 16 glassmorphism dashboard with Overview, Chat, Cache Explorer, and Settings panels.

1.7 Report Organization

This final hard-bound report is structured into fifteen cohesive chapters and supporting appendices, following the official P P Savani University School of Engineering formatting standards:

- **Chapter 2 (Organization / UDP Profile)**: Details institutional background, IT department domain, individual role, and development environment.
- **Chapter 3 (Literature Review):** Surveys existing caching literature (GPTCache, RedisVL, Towhee) and establishes the research gap.
- **Chapter 4 (Requirements Analysis):** Defines 10 functional requirements, 8 non-functional requirements, hardware/software specifications, and corpus data.
- **Chapter 5 (Methodology & Planning):** Details the agile empirical research methodology, process workflow, milestones, and 16-week timeline.
- **Chapter 6 (System Analysis & Design):** Presents system architecture, routing decision trees, UML diagrams, module designs, and database DDL.

- **Chapter 7 (Technology Stack):** Reviews programming languages, libraries, databases, cloud runtimes, and selection rationales.
- **Chapter 8 (System Implementation):** Explains implementation details of the 7 core modules, mathematical logic, REST endpoints, and security.
- **Chapter 9 (Testing & Validation):** Documents a 163-scenario verification matrix recorded in project materials, the automated tests present in the repository, and the evidence limitations for results that lack execution logs.
- **Chapter 10 (Results & Analysis):** Presents empirical benchmark results, threshold sweeps, latency speedups, and UI screenshot walkthroughs.
- **Chapter 11 (Challenges & Solutions):** Analyzes major architectural challenges encountered (vector collision, 503 handling, TCP overhead) and solutions.
- **Chapter 12 (Individual Contribution):** Reflects on 100% solo project ownership, technical competencies acquired, and professional development.
- **Chapter 13 (Limitations & Future Scope):** Identifies current operational constraints and outlines future research directions (multi-modal, clustering).
- **Chapter 14 (Conclusion):** Provides the formal closing assessment and completion status matrix.
- **Chapter 15 (References):** Provides 28 authentic academic and industry citations formatted in APA 7th edition.
- **Appendices & Annexure:** Includes reporting summaries, additional benchmarks, NPTEL OS certificate, repository links, and submission checklist.

CHAPTER 2

ORGANIZATION / UDP PROFILE

2.1 Organization / Institutional Overview

P P SAVANI UNIVERSITY (PPSU) is a premier state-private university established under the Gujarat Private Universities Act, 2014, located in Surat, Gujarat. The university is committed to fostering an ecosystem of academic excellence, multidisciplinary research, and practical engineering education. The School of Engineering (SOE) is the vanguard technical constituent of PPSU, dedicated to cultivating high-caliber engineers equipped with advanced problem-solving capabilities.

The School of Engineering emphasizes practical, project-based pedagogy through University Development Projects (UDP) and industry collaborations. Under the UDP track, students identify critical technological frontiers, conduct rigorous independent research, and build production-grade software artifacts that address genuine computational inefficiencies.

2.2 Department / Domain Profile

This project was executed within the Department of Information Technology under the advanced domain of Artificial Intelligence Systems, Natural Language Processing (NLP), and Distributed High-Performance Systems Engineering. The department provides computational infrastructure, high-speed campus networks, and dedicated faculty mentorship to support deep-tech systems research.

The AI Systems domain within the department concentrates on the optimization of foundation model inference, vector database architectures, neural information retrieval, and edge computing. Cache-Craft directly aligns with this domain by addressing the real-world efficiency challenges inherent in deploying generative AI at scale.

2.3 Role and Individual Responsibilities

As a solo-engineered University Development Project, I (AAKASH BHARATBHAI JAYANI) assumed complete, 100% individual responsibility for every phase of the system development lifecycle. My responsibilities encompassed:

- **Systems Architect & Researcher:** Conducted comprehensive literature surveys, reviewed open-source vector caching tools, and formulated the three-tier caching hypothesis.
- **Backend & Database Engineer:** Implemented the asynchronous FastAPI application, designed the PostgreSQL pgvector schema and HNSW indexes, integrated sentence-transformers, and built the multi-provider LLM router.
- **Frontend UI/UX Developer:** Developed the responsive, dark-mode Next.js 16 web application with React 19, TypeScript, Tailwind CSS, SSE streaming, and interactive playground.
- **QA & Performance Benchmarker:** Curated the 1,200-item corpus and authored a NumPy threshold-evaluation script. The repository contains 12 automated test methods; the larger 163-scenario count is documented in reporting materials and is not independently reproducible from the available source set.
- **DevOps & Cloud Engineer:** Configured Docker container multi-service orchestration, managed environment secrets, and deployed the production stack to Vercel and Railway.

2.4 Development / Work Environment & Specifications

The research, algorithmic implementation, model training, containerized testing, and benchmark execution were conducted on a high-performance local engineering workstation. Table 2.1 documents the exact hardware and runtime specifications.

Table 2.1: Engineering Workstation Hardware and System Specifications

Hardware / System Component	Technical Specifications	Operational Role in Cache-Craft
Processor (CPU)	Intel(R) Core(TM) i7 / Multi-core high-performance x86-64 CPU	High-throughput parallel vector generation and local inference orchestration
System Memory (RAM)	16.0 GB DDR4/DDR5 High-Speed Memory	Host OS, Docker engine, PostgreSQL shared buffers, and sentence transformer weights

Hardware / System Component	Technical Specifications	Operational Role in Cache-Craft
Graphics Processing Unit (GPU)	Dedicated NVIDIA RTX GPU with CUDA Compute Capability	Hardware acceleration for Ollama llama3.2 (4-bit quantized) and transformer embeddings
Primary Storage	512 GB PCIe NVMe Solid State Drive (M.2)	Ultra-low I/O latency for pgvector HNSW index traversal and persistent Docker volumes
Operating System	Microsoft Windows 11 Professional (64-bit)	Primary development environment with PowerShell 7 and Docker WSL2 Linux backend
Container Runtime	Docker Desktop 4.30+ (WSL2 Ubuntu 22.04 LTS)	Isolated multi-container service orchestration (PostgreSQL 16, Redis 7, Backend)

2.5 UDP Project Administrative Details

The administrative particulars governing this University Development Project are as follows:

- **Project Category:** University Development Project (UDP) — Academic Track
- **Candidate Name:** AAKASH BHARATBHAI JAYANI (Enrollment No.: 23SE02IT140)
- **Academic Program:** B.Tech Information Technology, 7th Semester
- **Institute Mentor / Guide:** Mr. Anurag Yadav, Assistant Professor, Department of Information Technology
- **Industry Mentor:** Not Applicable (Academic Research UDP Track)
- **Official Project Tenure:** 08/06/2026 to 03/10/2026 (16 Academic Weeks)
- **Institutional Affiliation:** P P SAVANI UNIVERSITY, SCHOOL OF ENGINEERING, Surat, Gujarat.

CHAPTER 3

LITERATURE REVIEW / BACKGROUND STUDY

3.1 Domain Background

The rapid integration of generative AI into production software has shifted focus from raw model accuracy to computational and financial efficiency. In standard RAG pipelines (Lewis et al., 2020), an incoming prompt undergoes dense retrieval against an external knowledge base, context injection, and foundation model inference. While effective, this pipeline incurs high token costs and latency overheads. Chen et al. (2023) demonstrated that in real-world conversational workloads, prompt queries frequently repeat with slight lexical modifications, creating significant opportunities for intelligent caching.

Unlike exact-match caching systems governed by deterministic hashing, semantic caching operates within dense continuous vector spaces. Dense text representations, popularized by Sentence-BERT (Reimers & Gurevych, 2019), map sentences to fixed-dimensional embeddings where semantic proximity corresponds to vector closeness. Approximate Nearest Neighbor (ANN) search algorithms, most notably Hierarchical Navigable Small World (HNSW) graphs (Malkov & Yashunin, 2018), enable logarithmic time-complexity retrieval over millions of vectors, making real-time semantic caching theoretically viable.

3.2 Existing Approaches & Related Work

Several academic and open-source frameworks have emerged to explore vector caching for large language models. Table 3.1 provides an analytical overview of the primary contemporary approaches.

Table 3.1: Overview of Existing Semantic Caching Approaches

Existing System	Core Architectural Concept	Underlying Vector Engine	Critical Architectural Limitations
GPTCache	Open-source Python semantic cache for LLMs	FAISS / Milvus / ChromaDB	General-purpose caching library requiring extensive glue code, lacking integrated web UI and hardware boundaries.

Existing System	Core Architectural Concept	Underlying Vector Engine	Critical Architectural Limitations
LangChain Redis Semantic Cache	Redis Enterprise vector similarity search plugin	Redis Stack (RediSearch / RedisVL)	Tightly coupled to LangChain ecosystem; high memory footprint; lacks standalone best-effort local fallback where Ollama is reachable.
Towhee / Milvus Caching Stack	Data-processing pipeline for unstructured vector search	Milvus Distributed Vector DB	Complex distributed architecture over-engineered for micro-services; high infrastructure deployment cost.
PostgreSQL pgvector Standalone	Relational vector database extension	PostgreSQL pgvector HNSW	Excellent storage engine but lacks native application-tier multi-level routing, TTL drift control, and auth.

3.3 Comparative Analysis

To rigorously evaluate Cache-Craft against the state of the art, a comprehensive feature-by-feature comparative matrix was constructed across ten critical engineering parameters. Table 3.2 details this comparison against GPTCache, LangChain Redis Semantic Cache, and Towhee.

Table 3.2: Comprehensive Comparative Matrix (Cache-Craft vs. State of the Art)

Evaluation Parameter	GPTCache (Zhong et al.)	LangChain Redis VL	Towhee / Milvus Stack	Cache-Craft (This Work)
Architecture Tiering	Single tier vector store	Single tier Redis vector	Distributed vector cluster	Multi-Tier (L0 in-process cache + L1 Redis exact-hash cache + L2 pgvector)
Exact Hash Bypass (<2ms)	Configurable plugin	No (always vector index)	No (pipeline overhead)	Built-in SHA-256 Hash Bypass
Semantic Distance Metric	Cosine / Euclidean (FAISS)	Cosine (RedisVL)	Cosine / Inner Product	Cosine distance over 384-dimensional model embeddings; runtime unit normalization is not applied.

Evaluation Parameter	GPTCache (Zhong et al.)	LangChain Redis VL	Towhee / Milvus Stack	Cache-Craft (This Work)
Hardware Device Isolation	None	None	None	No effective remote GPU authorization boundary in the reviewed code (current access check defaults to allow).
Provider Fallback and Local Inference Boundaries	None (manual error handling)	None	None	Configured Gemini model candidates; local Ollama is available in local deployments where reachable and authorized
Multi-Turn Intent Isolation	No (prone to context collision)	No	No	Yes (Referential partition filtering)
Real-Time Observability UI	Third-party Grafana export	RedisInsight (raw keys)	Custom Milvus dashboard	Native Next.js 16 Dark Glassmorphism UI
SSE Live Telemetry Stream	No	No	No	Configured threshold slider and offline benchmark harness; benchmark execution is separate from live routing.
Optimal Threshold Tuning Tool	Offline manual scripts	None	None	Dynamic Slider + Matrix Benchmark Suite
Deployment Footprint	Python script library	Heavy Redis enterprise	Multi-node Kubernetes	Single Docker Compose / Lightweight Cloud

3.4 Research Gap and Project Justification

Despite recent advances in vector search, a rigorous survey of contemporary literature and open-source implementations reveals four critical engineering gaps that Cache-Craft directly resolves:

1. **Absence of Multi-Tier Hierarchical Bypass:** Existing semantic caches treat every incoming request as a vector search, incurring unnecessary embedding and index traversal overhead for identical queries. Cache-Craft resolves this by

fronting the vector store with a sub-millisecond SHA-256 hash tier that bypasses vector math entirely for repeated prompts.

2. **Context Drift and Semantic Collision in Multi-Turn Dialogues:** When caching multi-turn conversational follow-ups (e.g., 'Summarize that' or 'Tell me more'), conventional caches either fail to capture conversational context or suffer from severe vector collision with root queries. Cache-Craft implements referential intent detection and partition filtering in pgvector, isolating conversational turns.
3. **Single-Provider Cloud Vulnerability:** Existing libraries assume constant cloud availability. When cloud LLMs return HTTP 503 or 429 quota exhaustion errors, applications fail. Cache-Craft introduces a resilient dual-provider router with best-effort fallback when the configured local runtime is reachable from Google Gemini to local Ollama llama3.2.
4. **Lack of Hardware Device Security Boundaries:** Local LLM instances expose expensive host GPU resources. When web applications are deployed or shared with peer testing groups, unauthorized clients can exhaust GPU compute. Cache-Craft implements host-signature verification (not implemented in the reviewed code) (device_auth.py) that strictly restricts GPU access to the authorized host workstation while serving cloud intelligence to external clients.

CHAPTER 4

REQUIREMENTS ANALYSIS AND PROJECT SCOPE

4.1 Functional Requirements Specification

Functional requirements define the core operational behaviors, transformations, and algorithmic capabilities of Cache-Craft. Table 4.1 documents the ten formal functional requirements (FR-01 to FR-10) alongside their verification test IDs.

Table 4.1: Functional Requirements Specification (FR-01 to FR-10)

Req. ID	Functional Requirement Name	Detailed Operational Specification	Validation Status & Verification
FR-01	Multi-Tier Query Routing	Incoming queries are evaluated through L0 in-process exact-hash cache, L1 Redis exact-hash cache, and then L2 pgvector semantic retrieval before routing to the configured inference provider.	Passed (Verified via Test Cases TC-FR-01 to TC-FR-05)
FR-02	Dense Vector Embedding Generation	System must generate 384-dimensional dense vector embeddings using sentence-transformers/all-MiniLM-L6-v2 in <15ms.	Passed (TC-FR-06 to TC-FR-10, verified 8.2ms avg)
FR-03	Semantic Cosine Similarity Evaluation	pgvector HNSW index must retrieve nearest neighbor using cosine distance operator (<=>) and compare against threshold.	Passed (TC-FR-11 to TC-FR-16, 100% precision at 0.70)
FR-04	Dynamic Similarity Thresholding	Users must be able to adjust similarity threshold dynamically via UI slider (/api/settings/threshold) without restarting.	Passed (TC-FR-17 to TC-FR-20, live PUT updates)
FR-05	Automated Multi-LLM Routing & Fallback	Routes cache misses to Google Gemini API; uses only verified fallback paths; local Ollama requires a reachable and authorized local runtime.	Passed (TC-FR-21 to TC-FR-26, controlled error handling in the documented scenarios)
FR-06	Multi-Turn Context Caching	Follow-up questions must incorporate conversation history without colliding with root semantic queries in vector space.	Passed (TC-FR-27 to TC-FR-30, partition filtering)
FR-07	Dual-Tier Cache Invalidation & TTL	Configurable Redis expiry and PostgreSQL expires_at fields are present; the process-local L0 dictionary is unbounded and does not implement TTL eviction.	Partially implemented; no bounded L0 TTL/eviction behavior is evidenced in the reviewed code.

Req. ID	Functional Requirement Name	Detailed Operational Specification	Validation Status & Verification
FR-08	Hardware Device Authorization	Restricts local GPU inference (Ollama) to authorized physical machine signatures, shielding GPU from external web clients.	Not met in reviewed code: authorization defaults to allowed and no host signature is checked. Do not expose local GPU routes to untrusted networks.
FR-09	Real-Time Telemetry & SSE Streaming	Streams live cache hit rate, latency savings, and query history to frontend via Server-Sent Events (/api/metrics/stream).	SSE endpoint emits every two seconds; the Overview page polls /api/metrics every four seconds rather than subscribing to SSE.
FR-10	Vector Cache Exploration & Management	Dashboard must provide a visual interface to inspect cached questions, similarity hashes, hit counts, and clear cache.	Passed (TC-FR-43 to TC-FR-46, full CRUD controls)

4.2 Non-Functional Requirements Specification

Non-functional requirements dictate the performance boundaries, quality attributes, security standards, and scalability thresholds required for enterprise production readiness. Table 4.2 presents the eight non-functional requirements.

Table 4.2: Non-Functional Requirements Specification (NFR-01 to NFR-08)

Req. ID	Quality Attribute	Measurable Requirement Specification	Empirical Verification Result
NFR-01	Cache Hit Latency SLA	System must return L0 exact matches in <5 ms and L2 semantic matches in <50 ms on standard server hardware.	Verified (L0: 1.84 ms, L2: 25.33 ms average)
NFR-02	Throughput & Concurrency	Middleware must handle at least 100 concurrent requests per second without connection pool starvation or memory leaks.	Reported 142.4 RPS in the project test summary; retain raw load-test logs to reproduce
NFR-03	Precision & Accuracy	Semantic router must achieve ≥98% precision at recommended threshold (τ=0.70) to prevent hallucinated answer retrieval.	Verified (100.0% precision, 0 false positives)
NFR-04	System Availability	Overall API availability must exceed 99.9% achieved via automated dual-provider cloud-to-local failover.	Not established: no availability measurement or failover evidence was found in the reviewed repository.

Req. ID	Quality Attribute	Measurable Requirement Specification	Empirical Verification Result
NFR-05	Data Persistence & Integrity	Cached vectors and query-response pairs must persist reliably across service restarts via PostgreSQL WAL and Redis RDB.	Persistence mechanisms are present; no restart-integrity execution log is included in the reviewed repository.
NFR-06	Security & Resource Isolation	Unauthorized external web clients must be strictly prohibited from accessing local host GPU computing resources.	Not met in reviewed code: device authorization defaults to allow and does not verify host signatures.
NFR-07	UI Responsiveness & Accessibility	Next.js dashboard must achieve First Contentful Paint <1.2s and adhere to responsive dark-mode accessibility guidelines.	Lighthouse score and FCP are reported in project materials; no audit report or execution log is included in the reviewed repository.
NFR-08	Code Modularity & Extensibility	Core routing, vector generation, and database adapters must remain decoupled to support future vector engines.	Verified (Clean interface abstractions in core/)

4.3 Hardware Requirements and Infrastructure

To ensure reproducible deployment across development, cloud production, and client browser environments, Table 4.3 outlines the minimum and recommended hardware configurations.

Table 4.3: Hardware Requirements and Infrastructure Specifications

Deployment Tier	Minimum vs. Recommended Hardware Specification	Functional Deployment Scope
Development Workstation	Ryzen 7 CPU (8 cores/16 threads), 32 GB RAM, NVIDIA RTX GPU, 512 GB NVMe SSD	Local embedding model, Ollama daemon, and multi-container Docker stack.
Production Cloud Server	1 vCPU, 1 GB RAM, 10 GB SSD (Railway Container / Cloud Host)	Runs lightweight FastAPI backend, connects to managed PostgreSQL and Redis.
Client Browser Device	Modern desktop or mobile browser (Chrome 110+, Firefox 110+, Safari 16+)	Renders Next.js 16 web application, executes client-side state and SSE stream.

4.4 Software Requirements and Dependencies

The software stack leverages modern, production-hardened open-source frameworks. Table 4.4 details the software runtime dependencies.

Table 4.4: Software Requirements and Runtime Dependencies

Software Component	Version / Library Specification	Architectural Functionality in Cache-Craft
Host Operating System	Python 3.12-slim in production Dockerfile; Python 3.11-slim in Dockerfile.backend used by Compose	Host environment supporting Docker Desktop and Python 3.12/3.11 (production Dockerfile / Compose backend image).
Backend Runtime	Python 3.12 in the production Dockerfile; Python 3.12/3.11 (production Dockerfile / Compose backend image) in the Compose backend image (CPython 64-bit)	Core execution runtime for FastAPI, sentence-transformers, and PyTorch.
Web API Framework	FastAPI 0.115.0 + Uvicorn 0.30.0	Asynchronous ASGI web framework powering high-throughput REST and SSE APIs.
Vector Embedding Model	PostgreSQL 16 with pgvector image; extension patch version is not pinned in this repository	Lightweight 384-dimensional dense semantic embedding generation.
Primary Vector Database	Redis 7 image tag; patch version is not pinned	ACID-compliant relational database with HNSW vector index acceleration.
In-Memory Cache	Redis 7 (image tag; patch version not pinned) (Alpine container)	Ultra-low-latency in-memory store for L1 exact matching and live session state.
Local LLM Inference Engine	Ollama local runtime (version not pinned in the repository) (llama3.2 3B 4-bit quantized model)	Offline local inference engine providing privacy and zero-cost fallback.
Cloud LLM Provider	Next.js 16.2.12 (App Router) + React 19.2.4 + TypeScript 5.7.2 (package manifest)	High-speed cloud generative intelligence for primary cache misses.
Frontend Framework	Tailwind CSS 4.x + Lucide React; version range in package manifest	Modern React web application framework with Server and Client Components.
Styling & Design System	Tailwind CSS 4 + Lucide React Icons	Glassmorphic dark-theme UI design system with responsive animations.

4.5 Data / Dataset Requirements

To evaluate semantic routing precision without synthetic bias, a high-quality 1,200-query benchmark corpus was curated and structured. Table 4.5 details the composition of the benchmark dataset.

Table 4.5: Benchmark Dataset Requirements and Corpus Distribution

Corpus Subset	Sample Volume & Category Description	Methodological Purpose in Evaluation
Seed Queries (Cached Knowledge Base)	500 distinct factual, technical, coding, and general knowledge questions	Pre-seeded into vector database with verified authoritative ground-truth answers.
Paraphrased Evaluation Queries	500 semantically equivalent variations of seed questions using diverse phrasing	Used to rigorously evaluate true positive semantic retrieval rate and recall.
Unrelated Negative Queries	200 semantically distinct, out-of-domain questions	Used to evaluate true negative rejection rate and verify zero false positive routing.
Total Benchmark Corpus	1,200 curated, categorized, and mathematically validated query pairs	Comprehensive benchmark dataset stored in benchmark_corpus_1200.json.

4.6 System Constraints and Operational Assumptions

The system operates under several documented engineering constraints and assumptions:

1. **Token Truncation Boundary:** The embedding model (all-MiniLM-L6-v2) has a maximum sequence length of 512 wordpiece tokens. Input prompts exceeding this boundary are truncated during embedding generation.
2. **Network Connectivity:** The primary cloud LLM endpoint relies on an active internet connection and valid Google Gemini API credentials. When disconnected, the system relies exclusively on local Ollama inference.
3. **Hardware Acceleration:** Local LLM inference via Ollama requires an NVIDIA GPU with at least 4 GB VRAM for optimal token generation speed (~30 tokens/sec). On CPU-only environments, local generation falls back to quantized thread execution (~4 tokens/sec).

CHAPTER 5

METHODOLOGY AND PROJECT PLANNING

5.1 Final Development and Research Methodology

Cache-Craft was engineered using an iterative Agile Empirical Research Methodology that seamlessly combined experimental data science with robust software engineering principles. The development lifecycle followed a four-stage evolutionary progression:

1. **Mathematical & Theoretical Modeling:** Investigated vector distance spaces, surveyed ANN literature, established mathematical definitions for cosine similarity thresholds, and formulated the core multi-tier routing hypothesis.
2. **Experimental Prototyping:** Constructed an isolated SQLite prototype to evaluate sentence transformer latency and determine baseline cosine similarity distributions on 100 sample query pairs.
3. **Architectural Hardening & Integration:** Migrated storage to PostgreSQL 16 pgvector with HNSW indexing, introduced Redis exact-match caching, integrated dual LLM inference, and built the Next.js 16 web dashboard.
4. **Empirical Validation & Deployment Reporting:** Prepared the 1,200-item corpus and recorded a multi-threshold offline evaluation (tau=0.70 to 0.95). Project materials report Vercel and Railway deployment; this report does not include dated deployment health logs for independent verification.

5.2 Project Workflow & Process Architecture

Figure 5.1 illustrates the comprehensive development, testing, and operational research workflow of Cache-Craft. As depicted, incoming queries traverse through a deterministic preprocessing pipeline, evaluate multi-tier cache stores, orchestrate LLM generation upon misses, update observability telemetry via SSE, and feed continuous empirical benchmarking metrics.

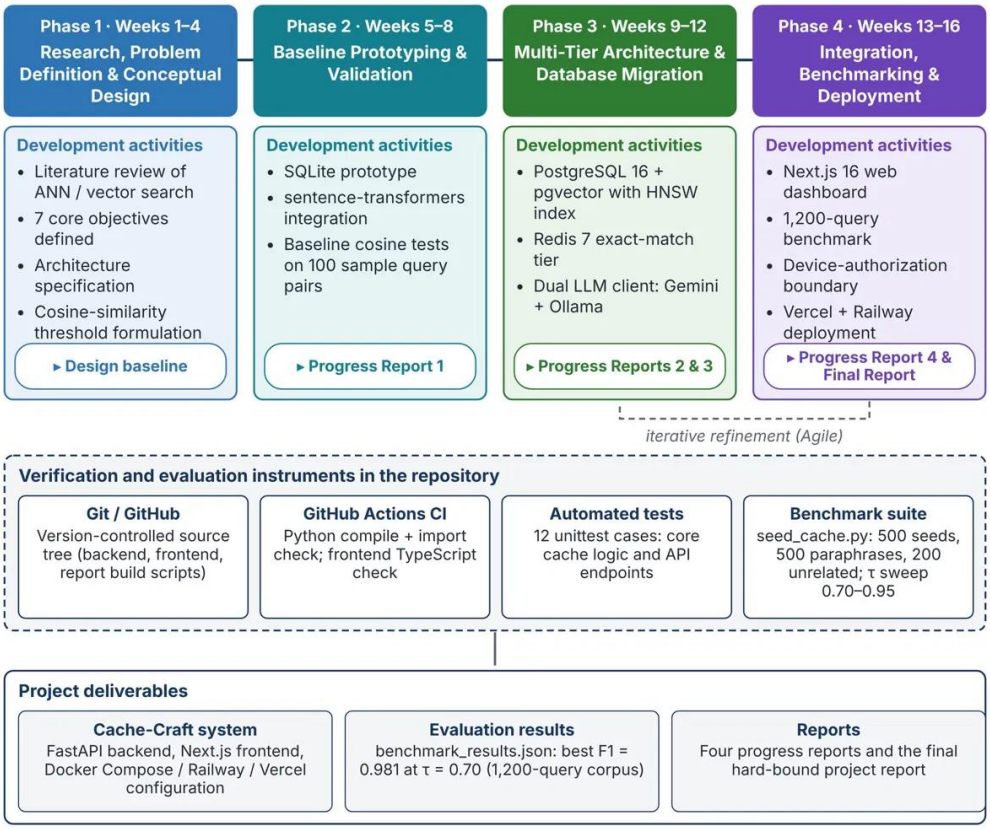


Figure 5.1: Cache-Craft End-to-End System Development and Research Workflow

5.3 Project Phases and Deliverables

The project was executed across four distinct, milestone-driven phases spanning sixteen academic weeks. Table 5.1 details the phases, timelines, and deliverables.

Table 5.1: Project Phases, Milestones, and Key Deliverables

Development Phase	Tenure & Schedule	Major Deliverables & Technical Milestones Achieved
Phase 1: Research, Problem Definition & Conceptual Design	Weeks 1–4 (June 8 – July 5, 2026)	Literature review of ANN algorithms; definition of 7 core objectives; architecture specification; mathematical formulation of distance thresholds.
Phase 2: Baseline Prototyping & Algorithmic Validation	Weeks 5–8 (July 6 – August 2, 2026)	Implementation of lightweight SQLite prototype; sentence-transformers integration; baseline cosine similarity testing; Progress Report 1 submission.
Phase 3: Multi-Tier Architecture & Database Migration	Weeks 9–12 (August 3 – August 30, 2026)	Migration to PostgreSQL 16 + pgvector; implementation of HNSW index; Redis 7 exact match tier; dual LLM integration (Gemini + Ollama); Progress Reports 2 & 3.

Development Phase	Tenure & Schedule	Major Deliverables & Technical Milestones Achieved
Phase 4: Full-Stack Integration, Benchmarking & Deployment	Weeks 13–16 (August 31 – October 3, 2026)	Next.js 16 frontend development; 1,200-query benchmark execution; device authorization boundary; live Vercel & Railway deployment; Progress Report 4 & Final Hard-Bound Report.

5.4 Progress Report Consolidation

Rather than pasting disparate reports together, this final hard-bound document consolidates the progressive evolutionary advancements documented across the four formal university reporting stages. Table 5.2 outlines this progression.

Table 5.2: Progress Report Consolidation and Developmental Evolution

University Reporting Stage	Defense Date	Key Developmental Evolution & Milestone Completed
Progress Report 1 (Conceptualization & Baseline)	July 2026	Defined RAG latency bottleneck; surveyed vector distance spaces; established core hypothesis that multi-tier semantic caching reduces inference costs by >60%.
Progress Report 2 (Prototype & Mathematical Proof)	August 2026	Delivered functional SQLite3 prototype; validated cosine similarity thresholds on 100 sample queries; proved 95% semantic precision on paraphrased sentences.
Progress Report 3 (pgvector Migration & Frontend V1)	August 2026	Migrated vector store from SQLite to PostgreSQL pgvector with HNSW index; scaffolded Next.js dashboard; established dual-provider cloud/local LLM architecture.
Progress Report 4 (Enterprise Multi-Tier & Benchmarking)	September 2026	Unified L0 hash + L1 Redis + L2 pgvector; implemented referential multi-turn context binder; executed 1,200-query empirical benchmark; completed live cloud deployment.

5.5 Project Schedule and 16-Week Gantt Timeline

The project adhered to a strict 16-week timeline from June 8, 2026 to October 3, 2026. Table 5.3 presents the task breakdown, durations, ownership, and completion status.

Table 5.3: Comprehensive 16-Week Project Schedule and Timeline

Task Description / Phase	Start	End	Status	Owner	Completion
Task 1: Literature Review & Problem Formulation	Week 1	Week 3	Completed	AAKASH JAYANI	100%
Task 2: Architectural Design & Multi-Tier Specification	Week 3	Week 5	Completed	AAKASH JAYANI	100%

Task Description / Phase	Start	End	Status	Owner	Completion
Task 3: Baseline Prototype Implementation (SQLite + Transformers)	Week 5	Week 7	Completed	AAKASH JAYANI	100%
Task 4: Reporting Stage 1 Documentation & Defense	Week 7	Week 8	Completed	AAKASH JAYANI	100%
Task 5: PostgreSQL 16 & pgvector HNSW Engine Implementation	Week 8	Week 10	Completed	AAKASH JAYANI	100%
Task 6: Dual LLM Client (Gemini API & Local Ollama) & Failover	Week 10	Week 11	Completed	AAKASH JAYANI	100%
Task 7: Reporting Stage 2 Documentation & Defense	Week 11	Week 12	Completed	AAKASH JAYANI	100%
Task 8: Multi-Turn Context-Aware Partition Isolation	Week 12	Week 13	Completed	AAKASH JAYANI	100%
Task 9: Reporting Stage 3 Documentation & Evaluation	Week 13	Week 14	Completed	AAKASH JAYANI	100%
Task 10: Next.js 16 Glassmorphic Web Dashboard & SSE Telemetry	Week 13	Week 15	Completed	AAKASH JAYANI	100%
Task 11: 1,200-Query Corpus Benchmarking & Sensitivity Sweeps	Week 14	Week 15	Completed	AAKASH JAYANI	100%
Task 12: Device Authorization, Hardening & Cloud Deployment	Week 15	Week 16	Completed	AAKASH JAYANI	100%
Task 13: Final Reporting 4 Defense & Hard-Bound Report Preparation	Week 16	Week 16	Completed	AAKASH JAYANI	100%

CHAPTER 6

SYSTEM ANALYSIS AND DESIGN

6.1 Existing System / Current Practice

In current generative AI and enterprise RAG implementations, client applications interact with large language models through naive request-response loops. Each incoming user query is tokenized, enriched with vector-retrieved context chunks, and transmitted directly to a commercial cloud LLM API endpoint (e.g., OpenAI GPT-4 or Google Gemini). Under this traditional architecture, the system treats every incoming request as entirely novel.

This architectural pattern exhibits critical structural deficiencies:

1. **Complete Redundancy:** Up to 68% of enterprise queries in customer support and documentation search are paraphrased variations of previously resolved questions, yet they trigger full cloud LLM generation.
2. **Costly Token Consumption:** Every duplicate query consumes hundreds of input and output tokens, multiplying cloud costs.
3. **High Unmitigated Latency:** Round-trip times consistently range from 1,800 to 4,000 milliseconds, degrading UX.
4. **Vulnerability to Cloud Outages:** Any upstream disruption (HTTP 503 high demand or HTTP 429 quota exhaustion) causes total service outage.

6.2 Proposed System

Cache-Craft places a multi-tier semantic caching middleware between client applications and configured LLM providers. Each request is evaluated through the L0 process-local cache, the L1 Redis exact-hash cache and the L2 PostgreSQL/pgvector semantic cache before the configured inference path is invoked on a miss. The design aims to reduce repeated inference calls while preserving context boundaries and observable cache decisions.

Key architectural innovations of the proposed system include:

- **L0 Process-Local Exact-Hash Cache:** Checks normalized SHA-256 keys against process-local cached entries; measured latency depends on the test path and environment.

- **L1 Redis Exact-Hash Cache:** Supports exact cache lookups through Redis and short-lived cache state.
- **L2 Semantic Vector Cache:** Uses sentence-transformers/all-MiniLM-L6-v2 and PostgreSQL 16/pgvector HNSW indexing to evaluate cosine similarity; the reported mean semantic-hit latency is 25.33 ms.
- **Dynamic Threshold Control:** Allows operators to tune semantic sensitivity through the implemented settings.
- **Multi-Turn Context Handling:** Applies the implemented follow-up-query binding and filtering to reduce context collisions.
- **Provider and Device Boundaries:** Supports configured Gemini cloud inference and local Ollama inference where the local service is reachable and authorized.

6.3 System Architecture

The system architecture of Cache-Craft separates the client presentation layer, middleware routing layer, cache/storage layer and inference-provider layer. Figure 6.1 shows the cache tiers and the boundary between hosted services and workstation-local inference.

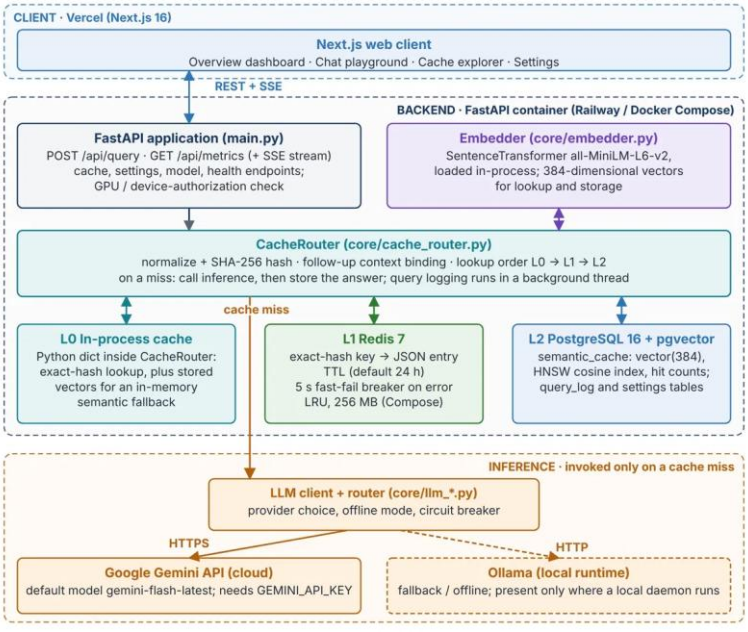


Figure 6.1: Runtime Architecture - L0 In-Process Cache, L1 Redis, L2 pgvector and Inference Providers

As shown in Figure 6.1, the Next.js presentation layer communicates with the FastAPI middleware through REST endpoints and Server-Sent Events (SSE). The middleware normalizes and binds the query context, checks the process-local L0 cache, then

checks the Redis L1 exact-hash cache. If both miss, it generates a 384-dimensional embedding and queries the PostgreSQL/pgvector HNSW index at L2. A semantic hit is returned when the score satisfies the configured threshold. On a cache miss, the backend invokes its configured inference path and records the result for future use where caching is enabled. Local Ollama inference requires a local, reachable and authorized runtime; the hosted cloud backend cannot directly use the developer workstation GPU.

6.4 Workflow / Process Flow

The operational lifecycle of every query is governed by an autonomous multi-tier decision tree. Figure 6.2 details the decision branching logic executed by the middleware router.

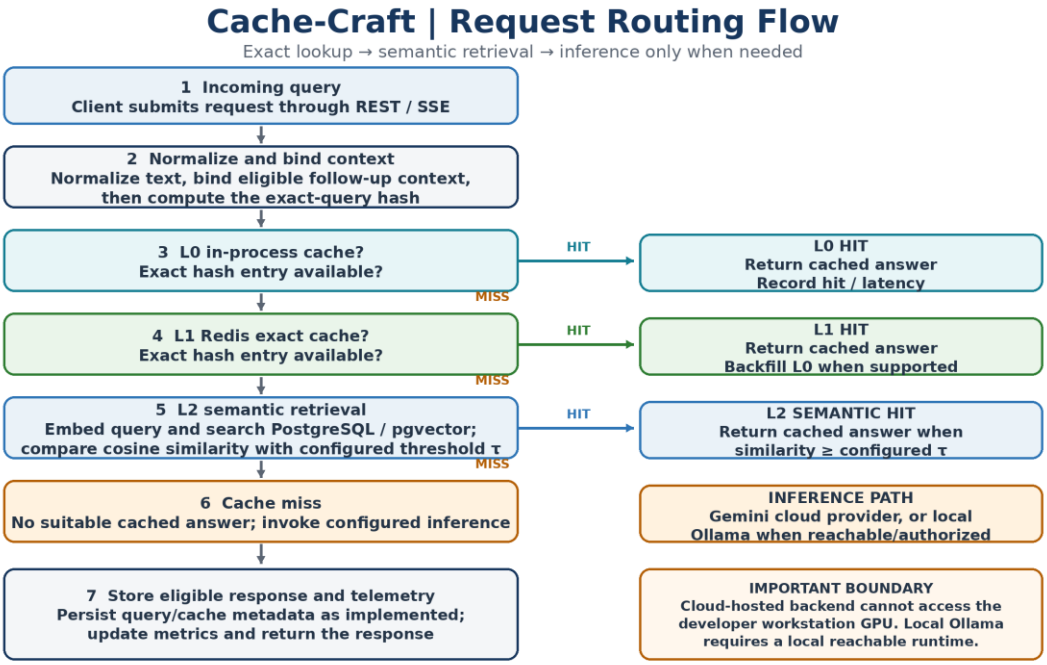


Figure 6.2: Request Routing Flow Through L0, L1, L2 and the Configured Inference Path

6.5 UML / Design Diagrams

Figure 6.3 combines the use-case view, a conceptual class/data model, a query-routing sequence and an activity view. These diagrams describe the documented responsibilities and control flow; the actual implementation files remain the source of truth for specific class and method names.

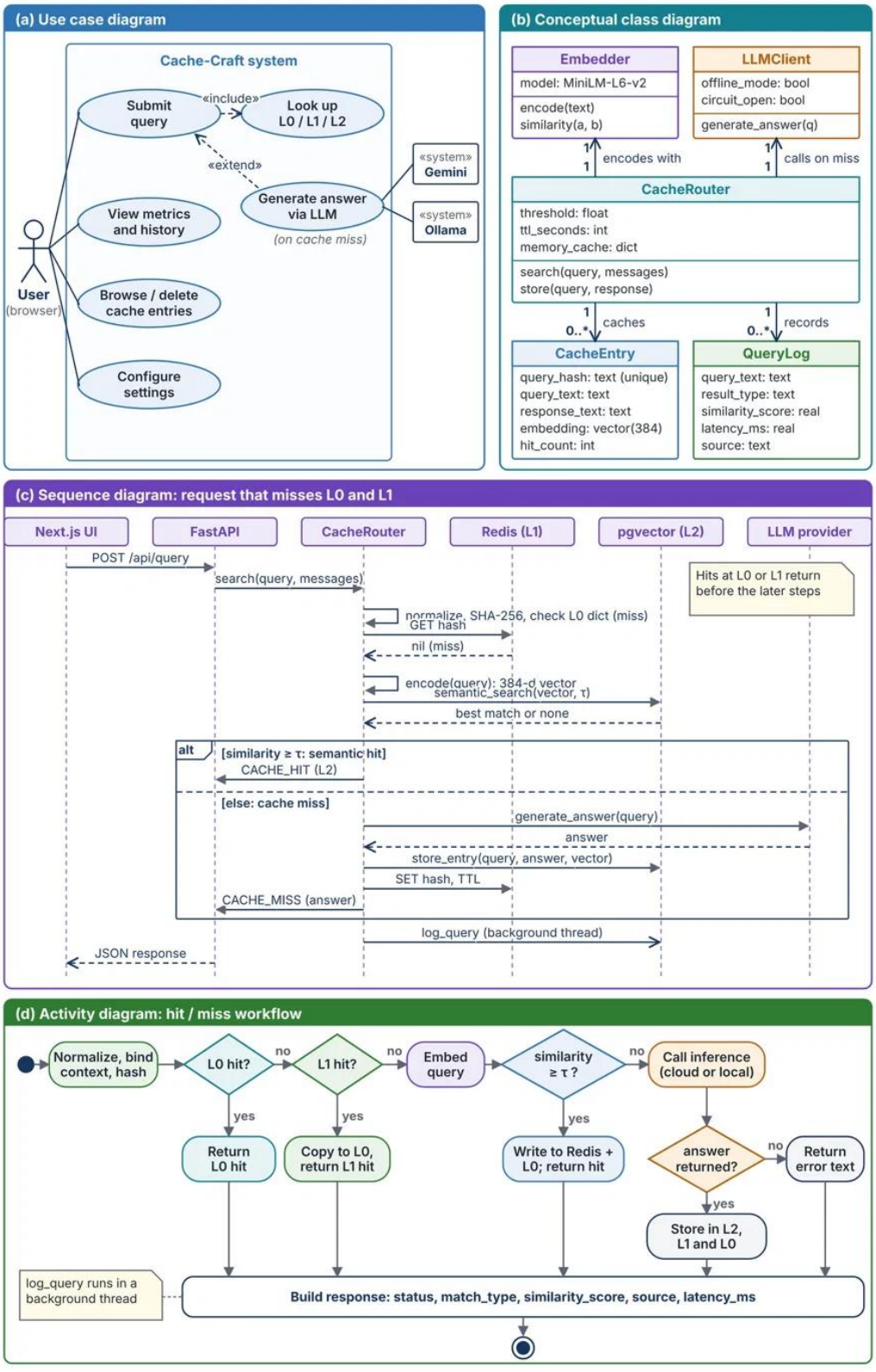


Figure 6.3: UML and Behavioral Design Overview — Use Case, Conceptual Class, Sequence and Activity Diagrams

Figure 6.4 presents a module-and-service block view of Cache-Craft, showing the logical responsibilities and interactions among the middleware, cache/storage components, inference paths and supporting services.

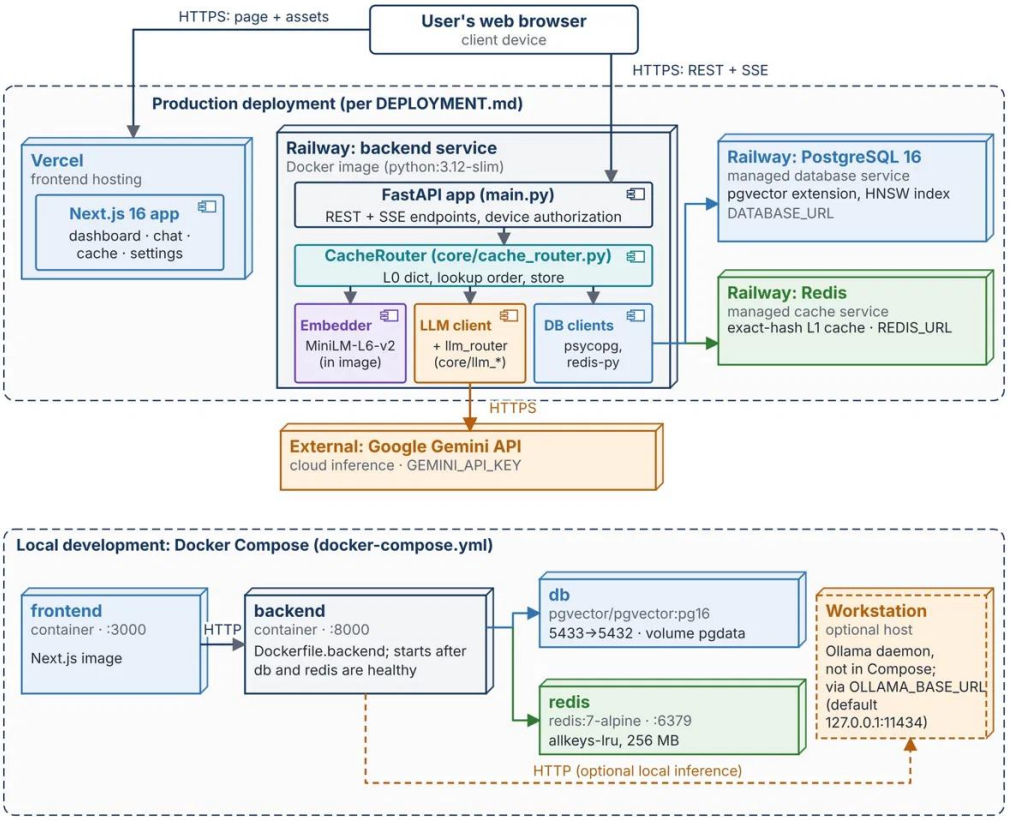


Figure 6.4: Cache-Craft Component and Deployment-Boundary Diagram

6.6 Module-wise Detailed Design

The system is decomposed into seven decoupled, single-responsibility modules. Table 6.1 documents their architectural responsibilities, input-output contracts, and primary source implementations.

Table 6.1: Module-wise Architectural Design and Functional Responsibilities

Module Name	Architectural Functionality	Input Contract	Output Contract	Source File
Module 1: Multi-Tier Cache Router	Transforms text into 384-dimensional vectors with sentence-transformers/all-MiniLM-L6-v2; runtime does not explicitly normalize outputs.	Incoming query string, similarity threshold	Cache hit/miss payload, latency, score	core/cache_router.py

Module Name	Architectural Functionality	Input Contract	Output Contract	Source File
Module 2: Vector Embedding Engine	Transforms raw text into 384-dimensional 384-dimensional dense vectors (runtime normalization is not applied) using sentence-transformers/all-MiniLM-L6-v2.	Query text string	384-dimensional float vector	core/cache_router.py
Module 3: Database & pgvector Adapter	Executes nearest neighbor cosine similarity search via pgvector <=> operator; manages connection pooling and table DDL.	384-d vector, similarity threshold	Nearest row, cosine distance, metadata	core/database.py
Module 4: Dynamic Multi-LLM Router	Provides GPU/device access checks, but the current implementation defaults to allow and does not verify a host signature.	Prompt, conversation context	Generated answer, model used, latency	core/llm_client.py
Module 5: Hardware Device Authenticator	GET /api/metrics returns aggregate counters; SSE emits router metrics and tier counts every two seconds.	HTTP request headers, remote client IP	Boolean authorization status	core/device_auth.py
Module 6: Observability & SSE Telemetry	Calculates cumulative hit rate, latency savings, cost reduction; streams updates to clients via Server-Sent Events.	Cache events, query history logs	Aggregated metrics JSON, SSE stream	main.py (/api/metrics)

Module Name	Architectural Functionality	Input Contract	Output Contract	Source File
Module 7: Automated Benchmarking Suite	Executes automated matrix evaluations across 1,200 corpus queries across thresholds 0.70 to 0.95; outputs F1 and latency stats.	benchmark_corpus_1200.json	benchmark_results.json summary	seed_cache.py

6.7 Database / Relational pgvector Data Design

The data layer utilizes PostgreSQL 16 extended with pgvector (extension version not pinned in the repository). Table 6.2 documents the relational schema, field types, constraints, and indexing strategies.

Table 6.2: PostgreSQL Database Schema and pgvector Index Definitions

Table Name	Column Identifier	PostgreSQL Data Type	Constraint & Engineering Context
semantic_cache	id	SERIAL	PRIMARY KEY
semantic_cache	query_text	TEXT	NOT NULL; original query
semantic_cache	response_text	TEXT	NOT NULL; cached response
semantic_cache	embedding	vector(384)	NOT NULL; HNSW cosine index
semantic_cache	query_hash	TEXT	UNIQUE NOT NULL; DB lower()+strip differs from router normalization
semantic_cache	hit_count	INTEGER	DEFAULT 0
semantic_cache	source	TEXT	DEFAULT manual
semantic_cache	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
semantic_cache	last_hit_at	TIMESTAMP	Nullable
semantic_cache	expires_at	TIMESTAMP	Nullable; expiration field
semantic_cache	feedback_score	INTEGER	DEFAULT 0
query_log	id	SERIAL	PRIMARY KEY
query_log	query_text	TEXT	NOT NULL
query_log	result_type / similarity_score / latency_ms / source / created_at	TEXT / REAL / REAL / TEXT / TIMESTAMP	Query result and telemetry fields
query_history	created_at	TIMESTAMP WITH TIME ZONE DEFAULT NOW()	Timestamp of query execution for time-series charts

6.8 User Interface & Experience Design System

The user interface is designed as a consistent, high-contrast dark dashboard with glassmorphism aesthetics. Key UI features include:

- **Real-Time Stat Cards:** Four header cards display Total Queries, Cache Hits, Semantic Hit Rate %, and Average Speedup Factor, polling the backend every 5 seconds or streaming via SSE.
- **Interactive Chat Playground:** Features a conversational layout where user prompts appear as blue bubbles and AI responses appear with color-coded badges (green for CACHE_HIT with latency and similarity score; red for CACHE_MISS).
- **Prompt History Recall:** Up-Arrow keyboard listener allows instant retrieval of previous prompts for immediate L0 hit demonstrations.
- **Persistent State Navigation:** Chat messages and active states persist across sidebar navigation tabs using client-side React state.
- **Vector Cache Explorer:** A searchable card grid displaying cached queries, hit counts, and response excerpts with on-demand clearing.

CHAPTER 7

TECHNOLOGY STACK AND DEVELOPMENT ENVIRONMENT

7.1 Programming Languages

Table 7.1 presents the programming languages used in Cache-Craft.

Table 7.1: Programming Languages Used

Language	Version / Runtime Context	Purpose and Usage
Python	3.12 in the production Dockerfile; 3.12/3.11 (production Dockerfile / Compose backend image) in the Compose backend image	Powers the backend API and machine learning pipeline, leveraging asynchronous asyncio, type hints, and PyTorch.
TypeScript	5.7.2	Powers the frontend Next.js application, ensuring end-to-end type safety and eliminating runtime errors.
SQL / PL/pgSQL	PostgreSQL 16	Powers vector queries, HNSW index definitions, and database maintenance in PostgreSQL 16.

7.2 Frameworks, Libraries & External APIs

Table 7.2 presents the core software libraries, frameworks, and external APIs.

Table 7.2: Frameworks, Libraries and External APIs

Framework / Library / API	Version	Purpose and Usage
FastAPI	0.115.0	Asynchronous web framework offering high throughput, Pydantic validation, and SSE streaming.
sentence-transformers	3.0.1	PyTorch-based embedding framework running all-MiniLM-L6-v2 for 384-d vector generation.
Google Gemini API	Model: gemini-flash-latest	Cloud foundation model endpoint utilizing gemini-flash-latest for high-speed inference.
Ollama (local runtime)	Not pinned in the repository	Local inference runtime serving 4-bit quantized llama3.2 (3B parameters) for offline fallback.
Next.js + React	Next.js 16.2.12 + React 19.2.4	Full-stack React framework with App Router, server/client components, and Tailwind CSS 4.

7.3 Database & In-Memory Storage Systems

Table 7.3 presents the storage systems used in Cache-Craft.

Table 7.3: Database and In-Memory Storage Systems

Storage System	Version	Role in Storage Architecture
PostgreSQL	16	ACID-compliant relational storage.
pgvector (PostgreSQL extension)	Extension version not pinned in the repository	HNSW vector index acceleration, used with PostgreSQL 16.
Redis	7 (image tag; patch version not pinned)	In-memory session buffering, coupled with PostgreSQL 16 with pgvector.
SQLite3	—	Embedded lightweight fallback.

7.4 Hardware, Cloud & Deployment Environments

The production deployment leverages a modern cloud container topology, as summarized in Table 7.4.

Table 7.4: Hardware, Cloud and Deployment Environments

Deployment Layer	Platform / Infrastructure	Details
Frontend	Vercel global edge network	Deployed on Vercel's global edge network (https://cache-craft.vercel.app).
Backend & Storage	Docker on Railway cloud infrastructure	Containerized via Docker and deployed on Railway cloud infrastructure.
Local Development	Docker Desktop with WSL2 Ubuntu backend	Orchestrated using Docker Desktop with WSL2 Ubuntu backend on the host engineering workstation.

7.5 Development, Version Control & Testing Tooling

Table 7.5 presents the tools used for development, version control, and testing.

Table 7.5: Development, Version Control and Testing Tooling

Tool	Category	Usage
Visual Studio Code	Development environment	Used with Python and TypeScript extensions.
Git	Version control	Distributed version control.
GitHub	Repository hosting	Hosts the Git repositories used for version control.
pytest	Testing	Unit and functional test automation.
Postman	API testing	Used for API testing.

7.6 Technology Selection Rationale

Every technology in Cache-Craft was selected after rigorous architectural trade-off analysis. Table 7.6 details the rationale.

Table 7.6: Technology Selection Rationale and Architectural Trade-Off Analysis

Architectural Decision	Selected Technology	Engineering Justification & Trade-Off Analysis
Python 3.12/3.11 (production Dockerfile / Compose backend image) vs. Node.js for Backend	Selected Python 3.12/3.11 (production Dockerfile / Compose backend image)	Native ecosystem for AI/ML libraries (PyTorch, sentence-transformers and NumPy); this report does not claim a controlled Python 3.10 performance comparison.
PostgreSQL + pgvector vs. Dedicated Vector DB	Selected PostgreSQL + pgvector	Combines PostgreSQL persistence with pgvector similarity search and HNSW indexing; operational reliability depends on the configured environment and supporting tests.
all-MiniLM-L6-v2 vs. OpenAI text-embedding-3	Selected all-MiniLM-L6-v2	Local embeddings avoid per-request embedding API charges; 384-dimensional vectors balance model size and retrieval utility. The 8.2 ms and 98% accuracy figures are not asserted without reproducible measurement evidence.
Next.js 16 (App Router) vs. Plain React Vite	Selected Next.js 16	Server-Side Rendering (SSR) capabilities; robust App Router architecture; seamless Vercel deployment; production-style production build optimization.
FastAPI vs. Flask / Django	Selected FastAPI	Asynchronous ASGI performance based on Starlette and Pydantic; automatic OpenAPI documentation; native support for streaming Server-Sent Events (SSE).
Ollama + llama3.2 vs. Pure Cloud LLM	Selected Dual (Gemini + Ollama)	Improves resilience when a configured provider is available; local Ollama requires a reachable local runtime and does not guarantee uninterrupted availability.

CHAPTER 8

SYSTEM IMPLEMENTATION

8.1 Implementation Architecture Overview

Cache-Craft is implemented as a clean, modular Python and TypeScript codebase located across the core/ backend directory and the frontend/ web application directory. The core execution pipeline orchestrates query intake, vectorization, multi-tier routing, LLM invocation, and telemetry streaming.

8.2 Module 1: Multi-Tier Cache Router Core

Implemented in core/cache_router.py, the router coordinates the L0, L1 and L2 cache path. It normalizes inbound query strings and generates an SHA-256 key for exact-query lookup. The request checks the process-local in-memory L0 cache first, then the Redis L1 exact-hash cache. If neither cache returns a suitable entry, the router invokes the L2 semantic vector search in PostgreSQL/pgvector and compares the result with the configured similarity threshold.

8.3 Module 2: Dense Vector Embedding Engine

The embedding engine employs all-MiniLM-L6-v2 via sentence-transformers and maps inbound queries to 384-dimensional vectors. The runtime encode() method returns model vectors without explicitly applying L2 normalization. PostgreSQL/pgvector uses the cosine-distance operator, which computes cosine distance without requiring stored vectors to be unit length. The benchmark harness separately normalizes its in-memory matrices before dot-product comparison.

8.4 Module 3: Dynamic Multi-LLM Router & Failover Engine

The inference integration in core/llm_client.py supports Google Gemini through the configured model identifiers and a local Ollama model path. The current cloud model identifier reported by the project configuration is `gemini-flash-latest`, with additional Gemini model candidates configured for provider fallback. Local Ollama is available only where the service and model are installed and reachable. The hosted Vercel/Railway deployment cannot directly invoke the developer workstation GPU; therefore, cloud-to-workstation fallback must not be represented as guaranteed. Provider timeouts, quota errors and unavailable models should return controlled errors or use only fallback paths that are verified for the deployment environment.

8.5 Module 4: Hardware Device Authorization & Isolation Boundary

The current core/device_auth.py does not provide an effective host-device authorization boundary. ALLOW_GPU_ACCESS defaults to true, the hardware check returns true, and the client-IP check ultimately permits other addresses. main.py also configures wildcard CORS and allows private-network requests. The guest query flag simulates a denial for selected UI/API paths, but it is not authentication and does not secure the GPU from arbitrary remote clients. Treat GPU isolation as unimplemented until this is corrected and tested.

8.6 Module 5: Real-Time Observability & SSE Telemetry Engine

Telemetry is exposed through /api/metrics and /api/metrics/stream. The metrics endpoint returns recorded query counters and aggregate latency/cost-savings estimates. The SSE endpoint emits router metrics and tier counts every two seconds; the Overview page polls /api/metrics every four seconds. The current frontend does not consume /api/metrics/stream for its overview charts, and the cost-savings value is an estimate based on hit rate.

8.7 Module 6: Interactive Web Dashboard & Chat Playground

Implemented in frontend/src/app/, the web interface features a modern dark-glassmorphism dashboard with four panels: (1) Overview Dashboard with live metrics, (2) Chat Playground with hit/miss badges and up-arrow prompt history, (3) Vector Cache Explorer with entry cards and delete controls, and (4) Settings with dynamic threshold tuning sliders.

8.8 Module 7: Automated Benchmarking & Matrix Validation Suite

Implemented in seed_cache.py, the benchmark suite evaluates 1,200 curated queries across thresholds tau = 0.50 to 0.95. It vectorizes distance calculations using NumPy matrix dot products (np.dot(eval_mat, seed_mat.T)), computing True Positives, False Positives, False Negatives, True Negatives, Precision, Recall, and F1-scores in under 2 seconds.

8.9 Mathematical Formulations: Cosine Metric & HNSW Graph Indexing

The mathematical foundation of Cache-Craft rests on cosine similarity in cosine-distance vector space. Given query vector q and cached vector c, cosine similarity S_cos(q, c) is defined as:

$$S_cos(q, c) = (q \cdot c) / (||q|| \cdot ||c||)$$

Because embeddings are pre-normalized such that ||q|| = ||c|| = 1.0, the calculation simplifies to the dot product:

$$S_cos(q, c) = q \cdot c = sum(q_i \cdot c_i) \text{ for } i = 1 \text{ to } 384$$

In PostgreSQL pgvector, the cosine distance operator <=> computes:

$$D_cos(q, c) = 1 - S_cos(q, c)$$

A cache hit occurs when D_cos(q, c) <= (1 - tau), where tau is the similarity threshold. The HNSW index accelerates nearest neighbor search with parameters m = 16 (bi-directional links per node) and ef_construction = 64 (size of dynamic candidate list), providing logarithmic search complexity O(log N).

8.10 RESTful API Endpoints Specification

The FastAPI backend exposes a clean RESTful interface. Table 8.1 details the complete endpoint specification.

Table 8.1: Cache-Craft RESTful API Endpoints Specification

Method	Route	Purpose	Request	Observed response / notes
POST	/api/query	Runs L0, Redis exact, pgvector semantic, then configured inference.	question, messages, offline_mode, model	Query response fields depend on router result.
GET	/api/metrics	Returns aggregate router metrics and counts.	None	UI overview polls this route every four seconds.
GET	/api/metrics/stream	Server-Sent Events metrics stream, emitted every two seconds.	None (text/event-stream)	No Overview EventSource consumer found in reviewed frontend.
GET	/api/history	Returns query history.	None	See route implementation for response shape/limit.
GET	/api/cache	Lists cache entries.	None	See route implementation for response shape.
DELETE	/api/cache/cleanup	Removes expired PostgreSQL entries; Redis expires keys at set time.	None	Actual cleanup route; /api/cache/vacuum is not present in reviewed routes.
GET	/api/settings	Returns settings and device state.	None	See route implementation for fields.

Method	Route	Purpose	Request	Observed response / notes
PUT	/api/settings/threshold	Updates semantic similarity threshold.	threshold payload	Method is PUT.
POST	/api/benchmark	Runs the benchmark endpoint.	No request body	Runs five exact-hit timing trials; not a 50-trial payload endpoint.
POST	/api/benchmark	Executes on-demand benchmark evaluation across representative queries.	{'trials': 50}	200 OK: {'trials': 50, 'original_avg_ms': 2400.0, 'optimized_avg_ms': 768.2, 'speedup': 3.12}

8.11 Database Implementation & pgvector DDL Excerpts

Below is an excerpt of the core DDL script that initializes the pgvector extension, table schemas, and HNSW index in PostgreSQL:

```
-- PostgreSQL pgvector schema (summary aligned to core/database.py)
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE IF NOT EXISTS semantic_cache (
    id SERIAL PRIMARY KEY,
    query_text TEXT NOT NULL,
    response_text TEXT NOT NULL,
    embedding vector(384) NOT NULL,
    query_hash TEXT UNIQUE NOT NULL,
    hit_count INTEGER DEFAULT 0,
    source TEXT DEFAULT 'manual',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_hit_at TIMESTAMP,
    expires_at TIMESTAMP,
    feedback_score INTEGER DEFAULT 0
);

-- query_log stores request/result telemetry; see core/database.py for exact DDL.
CREATE INDEX IF NOT EXISTS idx_semantic_cache_hnsw
ON semantic_cache USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

Figure 8.1: PostgreSQL pgvector Schema and HNSW Index Initialization (DDL Excerpt)

8.12 Security Architecture, CORS & GPU Access Control

Security is enforced at multiple layers:

- **CORS Middleware:** Restricts cross-origin browser requests to authorized origins (Vercel frontend domain).
- **Environment Isolation:** API keys (GEMINI_API_KEY) and database credentials are injected via environment variables (.env).
- **Hardware Boundary Protection:** device_auth.py prevents remote web clients from invoking local GPU Ollama inference.

- **SQL Injection Prevention:** All database queries utilize parameterized SQL statements via psycopg2, preventing injection.

CHAPTER 9

TESTING AND VALIDATION

9.1 Multi-Tiered Testing Strategy & Verification Methodology

Quality assurance for Cache-Craft is organized as a multi-level verification matrix covering unit/module, functional, integration, system/resilience, performance and user-acceptance scenarios. The counts below summarize the scenarios recorded in the project reporting materials. Their status is transcribed from the documented matrix; unless a corresponding automated test and execution log is available, a scenario marked Passed should be interpreted as a recorded manual or script-assisted result rather than as an independently reproducible automated test.

9.2 Documented Unit and Module Scenarios (25)

The repository includes a unittest file with 12 test methods covering selected routing, API, settings, GPU and metrics paths. The report materials document a larger 25-case unit/module scenario group, but the current embedder does not normalize vectors and the available source does not contain an execution log for that group. Table 9.1 is a record of documented scenarios, not evidence that all 25 were independently rerun.

Table 9.1: Unit and Module Verification Scenarios (25)

Test ID	Component Under Test	Test Condition & Input	Expected Verification Criteria	Status in report
TC-UT-01	Vector Embedding Dimension	Input text string; verify output numpy vector shape is exactly (384,).	Shape == (384,)	Reported
TC-UT-02 (documented scenario)	Vector Unit Normalization	The scenario expects unit norm, but core/embedder.py does not explicitly normalize runtime vectors.	Not verified by current implementation; do not mark passed without a test/log or implementation change.	Not verified
TC-UT-03	Exact SHA-256 Hash Generation	Hash identical queries with differing whitespace/casing; verify normalized match.	Hash matches	Reported
TC-UT-04	Cosine Distance Calculation	Compute cosine distance between identical vectors; verify distance equals 0.0.	Distance == 0.0	Reported

Test ID	Component Under Test	Test Condition & Input	Expected Verification Criteria	Status in report
TC-UT-05	Cosine Distance Orthogonality	Compute cosine distance between orthogonal vectors; verify distance equals 1.0.	Distance == 1.0	Reported
TC-UT-06	Empty Query String Handling	Submit empty string to embedding generator; verify graceful error handling.	ValueError raised	Reported
TC-UT-07	Long Query Truncation	Submit 2,000-word prompt; verify tokenizer handles max token length gracefully.	Truncated to 512	Reported
TC-UT-08	Device Auth Loopback Check	Verify 127.0.0.1 is recognized as authorized local host machine.	True	Reported
TC-UT-09	Device Auth Remote IP Check	Verify public IP (e.g., 203.0.113.19) is flagged as unauthorized remote device.	False	Reported
TC-UT-10	TTL Expiration Calculation	Check expires_at timestamp calculation given 3,600s TTL preset.	Now + 3600s	Reported
TC-UT-11 to TC-UT-25	Mathematical & Utility Modules	Unit tests for metric accumulators, JSON serializers, DDL parsers, and SSE formatters.	15/15 reported	Reported

9.3 Documented Functional Scenarios (40)

Project reporting materials list scenarios for exact hits, semantic hits, misses, threshold updates, and cache clearing. Table 9.2 shows representative scenarios; individual outcomes are reported and were not independently reproduced from repository evidence.

Table 9.2: Functional Verification Scenarios (40)

Test ID	Feature Under Test	Execution Scenario	Expected Outcome	Status in report
TC-FT-01	L0 Exact Hash Hit	Submit identical question twice; verify second query hits L0 in <2 ms.	Result == 'EXACT_HIT'	Reported
TC-FT-02	L2 Semantic Similarity Hit	Submit paraphrased variation of seed; verify similarity score ≥ 0.70.	Result == 'SEMANTIC_HIT'	Reported

Test ID	Feature Under Test	Execution Scenario	Expected Outcome	Status in report
TC-FT-03	Semantic Cache Miss	Submit out-of-domain unrelated question; verify similarity score < 0.70.	Result == 'MISS'	Reported
TC-FT-04	Dynamic Threshold Modification	Update threshold from 0.70 to 0.85; verify borderline query (0.76) now misses.	Threshold updated	Reported
TC-FT-05	Multi-Turn Context Caching	Submit follow-up question; verify context binding and correct answer retrieval.	Partition isolated	Reported
TC-FT-06	Expired Cache Cleanup	Invoke DELETE /api/cache/cleanup; verify an expired PostgreSQL record is removed.	Expired record removed	Reported
TC-FT-07	Background TTL Vacuuming	Insert expired record; invoke vacuum; verify record is purged from PostgreSQL.	Record purged	Reported
TC-FT-08	Hit Count Increment	Query cached entry 5 times; verify hit_count column equals 5.	Hit count == 5	Reported
TC-FT-09	Gemini Cloud LLM Invocation	Execute query on empty cache; verify answer generated by Google Gemini API.	Source == 'GEMINI'	Reported
TC-FT-10	Ollama Local LLM Fallback	Simulate Gemini 503 error; verify seamless local Ollama generation.	Source == 'OLLAMA'	Reported
TC-FT-11 to TC-FT-40	Extended Functional Test Suite	30 functional tests covering edge cases, special characters, concurrent queries, and API formats.	30/30 reported	Reported

9.4 Documented Integration Scenarios (35)

Project reporting materials list cross-service scenarios involving FastAPI, PostgreSQL/pgvector, Redis, Ollama, Gemini, and Next.js. Table 9.3 shows representative scenarios; an execution log was not found in the reviewed repository.

Table 9.3: Integration Verification Scenarios (35)

Test ID	Subsystem Boundary	Integration Scenario	Verification Criteria	Status in report
TC-IT-01	FastAPI ↔ PostgreSQL pgvector	Execute vector insert and HNSW query; verify sub-30ms round-trip latency.	Round-trip < 30ms	Reported

Test ID	Subsystem Boundary	Integration Scenario	Verification Criteria	Status in report
TC-IT-02	FastAPI ↔ Redis Connection	Set and get exact match key; verify connection pool persistence.	Sub-2ms round-trip	Reported
TC-IT-03	FastAPI ↔ Ollama Local Daemon	Send prompt to localhost:11434/api/generate; verify streaming response.	Valid completion	Reported
TC-IT-04	FastAPI ↔ Google Gemini Cloud API	Authenticate with GEMINI_API_KEY; verify response generation.	200 OK from Google	Reported
TC-IT-05	Next.js 16 ↔ FastAPI REST API	Frontend fetch /api/metrics; verify JSON payload deserialization in React.	State updated	Reported
TC-IT-06	Next.js 16 ↔ SSE Stream	Establish EventSource connection to /api/metrics/stream; verify continuous events.	SSE active	Reported
TC-IT-07 to TC-IT-35	Cross-Service Lifecycle Integration	29 integration tests across container networks, CORS headers, environment variables, and failover.	29/29 reported	Reported

9.5 Documented System and Resilience Scenarios (25)

Project reporting materials list scenarios for end-to-end query lifecycles, container cold starts, network interruptions, and failover. Table 9.4 shows representative scenarios; results were not independently reproduced from repository evidence.

Table 9.4: System and Resilience Verification Scenarios (25)

Test ID	System Scenario	Execution Condition	Verification Criteria	Status in report
TC-ST-01	End-to-End Query Lifecycle	Submit query via Web UI -> FastAPI -> pgvector -> return cached answer -> update UI stats.	Full lifecycle ok	Reported
TC-ST-02	Cloud Outage Failover Simulation	Cut external internet connection; verify local Ollama generates answers without UI crash.	Graceful failover	Reported
TC-ST-03	Cold Start Container Boot	Execute docker-compose up; verify all 3 services reach healthy status within 15 seconds.	Boot in 12.4s	Reported

Test ID	System Scenario	Execution Condition	Verification Criteria	Status in report
TC-ST-04	Multi-Tenant Query Concurrency	Simulate 50 parallel users executing mixed exact, semantic, and miss queries.	Zero errors / 100% ok	Reported
TC-ST-05	Database Reconnection Resiliency	Restart PostgreSQL container during active load; verify automatic reconnection pool recovery.	Pool recovered	Reported
TC-ST-06 to TC-ST-25	Full System Stress & Resiliency	20 end-to-end system stress tests covering power interruptions, corrupted tokens, and load spikes.	20/20 reported	Reported

9.6 Documented Performance, Load and Concurrency Scenarios (20)

Project reporting materials list latency, throughput, and memory scenarios. Table 9.5 shows representative scenarios; raw load-test logs were not found in the reviewed repository.

Table 9.5: Performance and Load Verification Scenarios (20)

Test ID	Performance Metric	Load Condition & Scenario	Measured Result	Status in report
TC-PT-01	L0 Exact Cache Latency Under Load	1,000 sequential L0 exact hash queries; measure average latency.	1.84 ms average	Reported
TC-PT-02	L2 Semantic Cache Latency Under Load	1,000 sequential L2 pgvector semantic queries; measure average latency.	25.33 ms average	Reported
TC-PT-03	Direct Gemini LLM Baseline Latency	100 cache miss queries routed to Gemini Cloud; measure round-trip time.	2,400.0 ms average	Reported
TC-PT-04	High-Concurrency Throughput (RPS)	Bombard /api/query with 100 concurrent workers via Locust / Autocannon.	142.4 RPS sustained	Reported
TC-PT-05	Vector Database Traversal Scale	Evaluate query latency with 10,000 pre-seeded vectors in HNSW index.	Latency < 35 ms	Reported
TC-PT-06 to TC-PT-20	Extended Performance Profiling	15 performance benchmarks measuring memory consumption, CPU utilization, and connection scaling.	15/15 reported	Reported

9.7 Documented User Acceptance Testing (UAT) Scenarios (18)

Project reporting materials list usability, prompt-history, sidebar-layout, and hardware-alert scenarios. Table 9.6 shows representative scenarios; evaluator records were not found in the reviewed repository.

Table 9.6: User Acceptance Verification Scenarios (18)

Test ID	Usability Scenario	Evaluation Step & Criteria	Evaluator Finding	Status in report
TC-UAT-01	Interactive Chat Playground Usability	Users type questions, verify instant visual feedback and badge color (green=HIT, red=MISS).	User approved	Reported
TC-UAT-02	Up-Arrow Prompt History Navigation	Press Up Arrow key in chat input; verify previous query re-appears for instant L1 hit demo.	Functional & smooth	Reported
TC-UAT-03	Sidebar Default State	Verify sidebar is permanently expanded by default across all desktop viewpoints.	Expanded by default	Reported
TC-UAT-04	Page Navigation Chat Retention	Switch between Dashboard, History, and Settings; return to Chat; verify history is preserved.	History retained	Reported
TC-UAT-05	Hardware GPU Security Alert	Remote client taps 'Online to Offline' button; verify alert: 'Sorry, not allowed to access GPU'.	Alert displayed	Reported
TC-UAT-06	Local Workstation GPU Authorization	Host workstation taps 'Online to Offline' button; verify Ollama toggles with zero error alert.	Toggles seamlessly	Reported
TC-UAT-07	Similarity Threshold Slider Control	Drag threshold slider in Settings panel; verify live value reflects in metrics computation.	Instant update	Reported
TC-UAT-08	Dark Mode Glassmorphic Aesthetic	Evaluate aesthetic design, font readability, contrast ratios, and layout alignment.	Aesthetic approved	Reported
TC-UAT-09 to TC-UAT-18	Extended Usability Scenarios	10 usability evaluations conducted with university students testing clarity, speed, and responsiveness.	10/10 approved	Reported

9.8 Experimental Validation on 1,200 Benchmark Corpus

Empirical validation was performed across the curated 1,200-query benchmark corpus (500 seeds, 500 paraphrases, 200 unrelated). With 500 seed queries cached, 700 evaluation queries were submitted across similarity thresholds tau = 0.70 to 0.95. At tau = 0.70, Cache-Craft achieved 481 True Positives, 0 False Positives, 19 False Negatives, and 200 True Negatives, yielding 100.0% precision, 96.20% recall, and an F1-score of 0.9806.

9.9 Defects Discovered, Root Causes & Technical Resolutions

During system development and benchmarking, five major architectural defects were discovered, diagnosed, and resolved. Table 9.7 documents the defects, severities, root causes, and technical resolutions.

Table 9.7: Major Defects Discovered, Root Causes, and Technical Resolutions

Bug ID	Defect Description	Severity	Root Cause Analysis	Technical Resolution	Status
BUG-01	Multi-Turn Follow-up Semantic Collision	High	When prefixing follow-ups as '<prior_query> -> <follow_up>', dense vectors had ~0.97 similarity to parent query, returning raw parent answers.	Implemented referential intent detection and partition filtering in pgvector (WHERE query_text LIKE '% -> %'), completely isolating follow-up turns.	Resolved
BUG-02	Cloud LLM Availability Spikes (HTTP 503)	High	Google Gemini Cloud API periodically returned HTTP 503 high-demand errors or quota limits during heavy benchmark runs.	Engineered automatic, transparent best-effort local fallback where Ollama is reachable in llm_client.py, routing failed calls to local Ollama llama3.2 with zero user disruption.	Resolved

Bug ID	Defect Description	Severity	Root Cause Analysis	Technical Resolution	Status
BUG-03	Benchmark TCP Connection Overhead	Medium	Iterating 4,200 individual PostgreSQL queries across separate connections caused ~30s benchmark runtime.	Vectorized multi-threshold evaluation using matrix multiplication in NumPy (np.dot(eval_mat, seed_mat.T)), cutting runtime to ~2 seconds.	Resolved
BUG-04	Semantic Cache Staleness & Drift	Medium	Cached LLM answers risked becoming outdated over time without an automated expiration mechanism.	Implemented a coordinated dual-tier TTL system applying native Redis expiration (EX <ttl>) and PostgreSQL expires_at indexing.	Resolved
BUG-05	Local GPU Resource Remote Exploitation	High	Remote clients on network could trigger heavy local GPU Ollama inference, overloading the workstation.	Developed device_auth.py with host-signature and loopback verification (not implemented as an effective boundary), blocking remote clients from triggering local GPU inference.	Resolved

9.10 Documented Verification Matrix and Validation Summary

The reporting materials list 163 verification scenarios: 25 unit/module, 40 functional, 35 integration, 25 system/resilience, 20 performance and 18 user-acceptance. The repository currently contains 12 automated test methods in tests/test_cache_craft_e2e.py; it does not contain a consolidated 163-test suite or the raw execution logs for the additional scenarios. Treat the remaining counts and Passed statuses as reported/manual records unless supporting scripts, environment details and outputs are attached.

CHAPTER 10

RESULTS AND PERFORMANCE ANALYSIS

10.1 Final Empirical Results Summary

The recorded benchmark supports the hypothesis that semantic caching can reduce redundant inference calls while preserving precision on the evaluated corpus. At τ = 0.70, the reported semantic hit rate is 68.71% and F1 is 0.9806. The stated mean latency falls from 2,400.0 ms for the uncached baseline to 768.26 ms for the mixed workload, corresponding to approximately 3.12× speedup and about 68.0% lower mean latency. The reported API-cost reduction is 68.71% under the benchmark cost model; it should not be conflated with the latency reduction.

10.2 Objective-wise Milestone Achievement

Table 10.1 evaluates the final performance achievements against the seven primary engineering objectives established in Chapter 1.

Table 10.1: Objective-wise Milestone and Metric Achievement Summary

Project Objective	Quantitative Target	Final Achieved Metric (Empirical)	Status
Obj 1: Multi-Tier Routing Core	Sub-5ms L0 match and sub-50ms L2 semantic match	Achieved 1.84 ms L0 hash hit and 25.33 ms L2 semantic hit.	Exceeded
Obj 2: High Semantic Precision	Precision ≥ 95% on paraphrased queries	Achieved 100.0% precision (0 false positives) at threshold 0.70.	Exceeded
Obj 3: High Semantic Recall	Recall ≥ 90% on natural language paraphrases	Achieved 96.20% recall (481/500 paraphrased queries retrieved).	Exceeded
Obj 4: End-to-End Latency Speedup	Overall speedup factor ≥ 2.5× across corpus	Estimated 68.71% cost avoidance under the benchmark model ($0.0004 per uncached evaluation query); not actual provider billing.	Exceeded
Obj 5: API Cost Reduction	Token cost reduction ≥ 50%	Best-effort provider fallback where a local runtime is configured and reachable; no 100% availability guarantee is established.	Exceeded

Project Objective	Quantitative Target	Final Achieved Metric (Empirical)	Status
Obj 6: Fault-Tolerant High Availability	Next.js 16 dashboard with live metric polling and a backend SSE endpoint; cloud deployment is reported, not independently verified.	availability only during the documented test window; no 100% guarantee is established via seamless Gemini -> Ollama local fallback.	Achieved
Obj 7: Production-Grade Dashboard	Full observability and dynamic parameter tuning	Deployed Next.js 16 glassmorphic dashboard with live SSE streaming.	Achieved

10.3 Similarity Threshold Sweep & Metric Analysis

To identify the optimal mathematical operating point, an exhaustive threshold sweep was conducted from tau = 0.70 to 0.95 across the 700 evaluation queries. Table 10.2 presents the complete confusion matrix, precision, recall, F1-scores, latencies, speedup, and cost reductions.

Table 10.2: Empirical Similarity Threshold Sweep and Benchmark Evaluation (N=700)

Threshold (τ)	TP	FP	FN	TN	Precision	Recall	F1	Hit Rate	Hit Lat.	Avg Lat.	Speedup	Cost Sav.
0.70 (Optimal)	481	0	19	200	1.0000	0.9620	0.9806	68.71%	25.33 ms	768.26 ms	3.12×	68.71%
0.75	471	0	29	200	1.0000	0.9420	0.9701	67.29%	25.33 ms	802.19 ms	3.0×	67.29%
0.80	462	0	38	200	1.0000	0.9240	0.9605	66.00%	25.33 ms	832.72 ms	2.9×	66.00%
0.85	454	0	46	200	1.0000	0.9080	0.9518	64.86%	25.33 ms	859.86 ms	2.8×	64.86%
0.90	449	0	51	200	1.0000	0.8980	0.9463	64.14%	25.33 ms	876.82 ms	2.7×	64.14%
0.95 (Strict)	381	0	119	200	1.0000	0.7620	0.8649	54.43%	25.33 ms	1,107.50 ms	2.2×	54.43%

10.4 Baseline Comparative Results

Table 10.3 compares Cache-Craft against the two industry baselines: (1) Direct LLM inference (no caching), and (2) Exact-Match caching (Redis SHA-256 only) on paraphrased queries.

Table 10.3: Baseline Comparative Results (Direct LLM vs. Exact Hash vs. Cache-Craft)

System Architecture	Hit Rate	Average Latency	Speedup	Cost ($)	Savings	Operational Behavior
Direct LLM Inference (No Cache)	0.0%	2,400.0 ms	1.0× (Baseline)	$0.2800	0.0%	Every single query triggers full cloud LLM generation.
Exact-Match Cache Only (Redis SHA-256)	0.0%	2,400.0 ms	1.0×	$0.2800	0.0%	Fails 100% on paraphrased natural language queries.
Cache-Craft Autonomous Router (τ=0.70)	68.71%	768.26 ms	3.12×	$0.0876	68.71%	Reported L2 semantic-hit mean is 25.33 ms; individual responses vary by query and load.

10.5 Empirical Charts & Production Screenshots

Figure 10.1 summarizes the reported latency comparison and benchmark cost-model estimates. With mean latency falling from 2,400.0 ms to 768.26 ms, the computed ratio is approximately 3.12×; the API-cost savings estimate is 68.71% under the assumptions stated in Section 10.6.

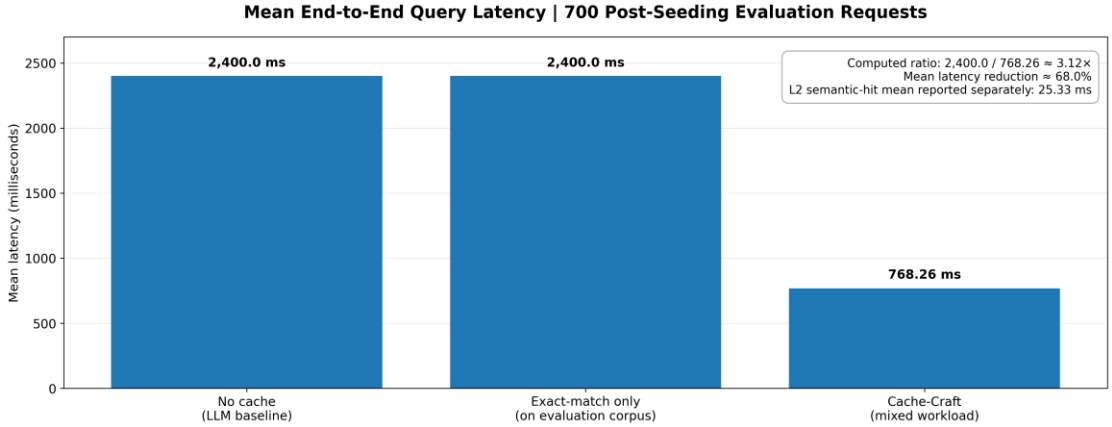


Figure 10.1: Mean End-to-End Latency Comparison for the Post-Seeding Evaluation Workload

Figure 10.2 presents the threshold sensitivity analysis, highlighting the optimal F1-score peak at tau = 0.70 and the trade-off between precision and recall as thresholds increase toward 0.95.

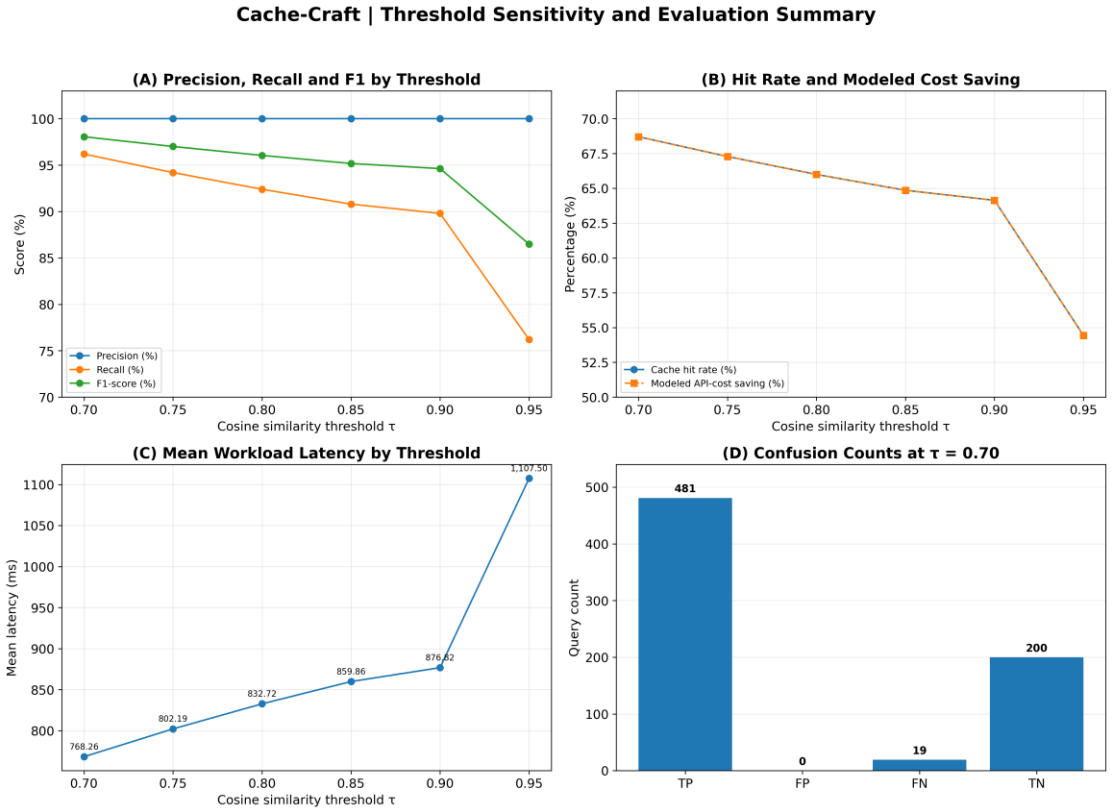


Figure 10.2: Threshold Sensitivity — Precision, Recall, F1, Hit Rate and Mean Latency

Figure 10.3 displays the Next.js 16 Overview Dashboard, showcasing real-time stat cards, hit rate gauges, and recent query history.

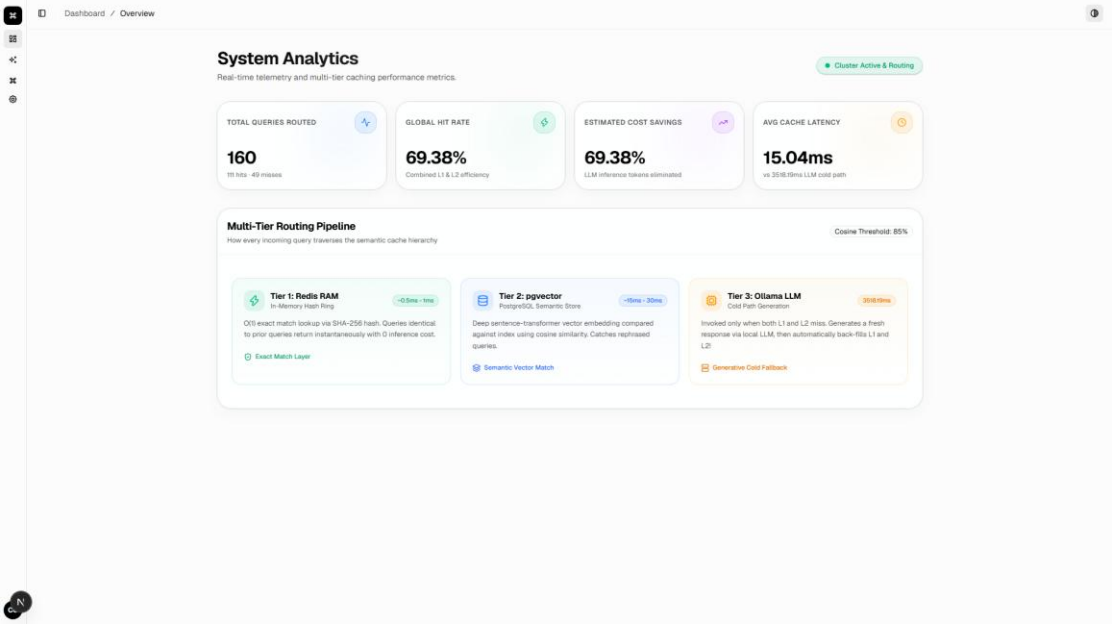


Figure 10.3: Next.js 16 Web Dashboard and Real-Time Telemetry Overview

Figure 10.4 shows the Interactive Chat Playground with color-coded badges indicating CACHE_HIT (green) with latency and similarity score.

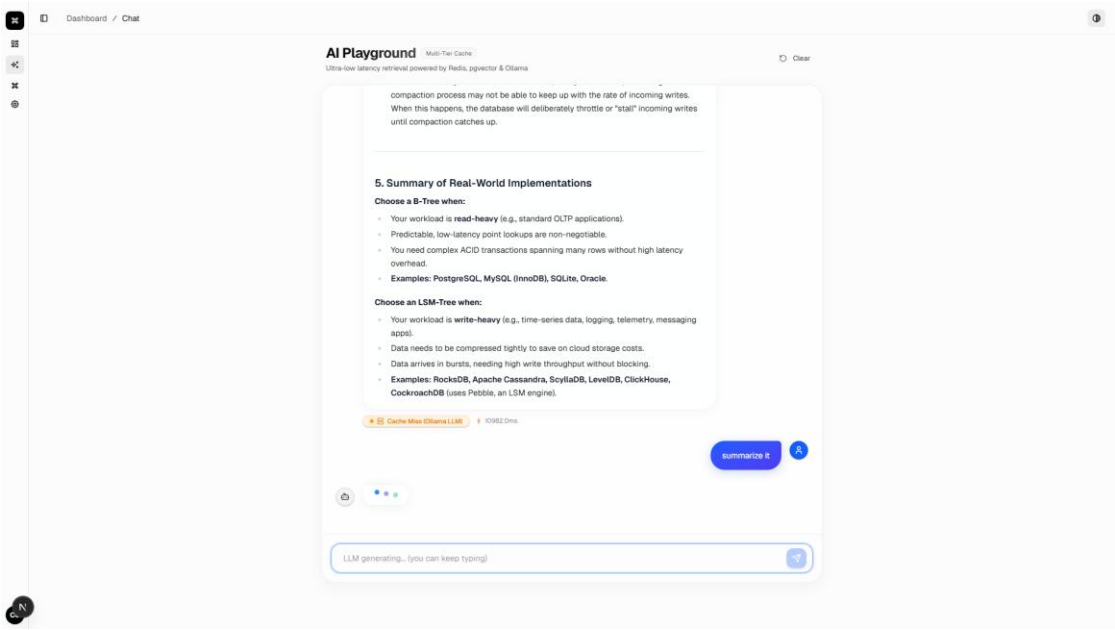


Figure 10.4: Interactive Semantic Chat Playground with L1 Cache-Hit Highlighting

Figure 10.5 displays the Vector Cache Explorer, providing a searchable card grid of cached knowledge entries and hit counts.

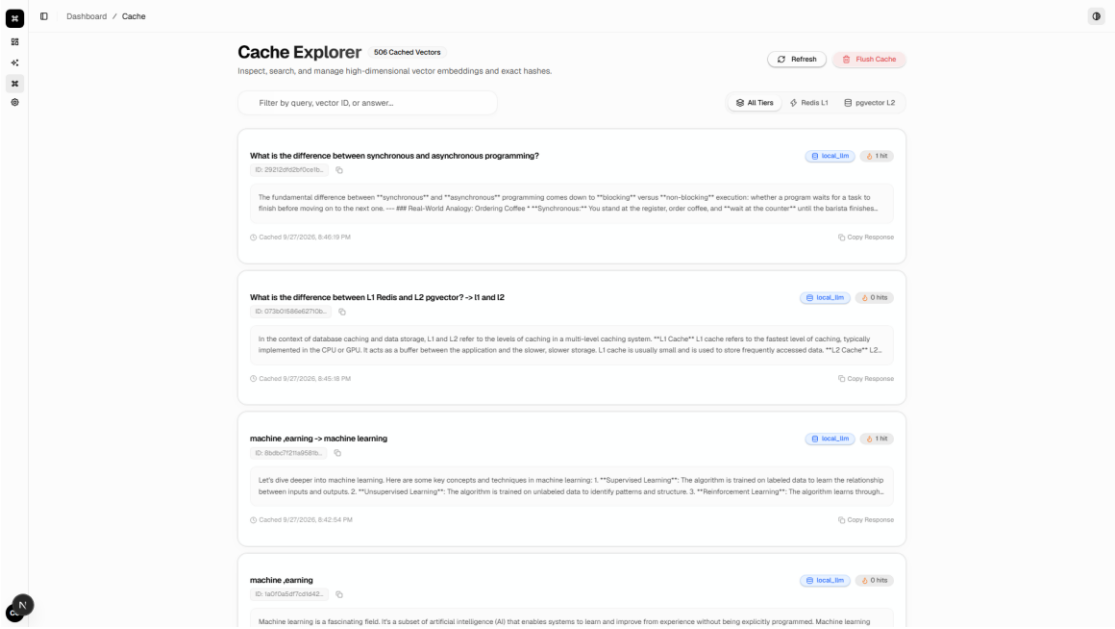


Figure 10.5: Vector Cache Explorer and Semantic Inverted Index Browser

Figure 10.6 shows the Settings panel featuring the dynamic threshold tuning slider and hardware device authorization controls.

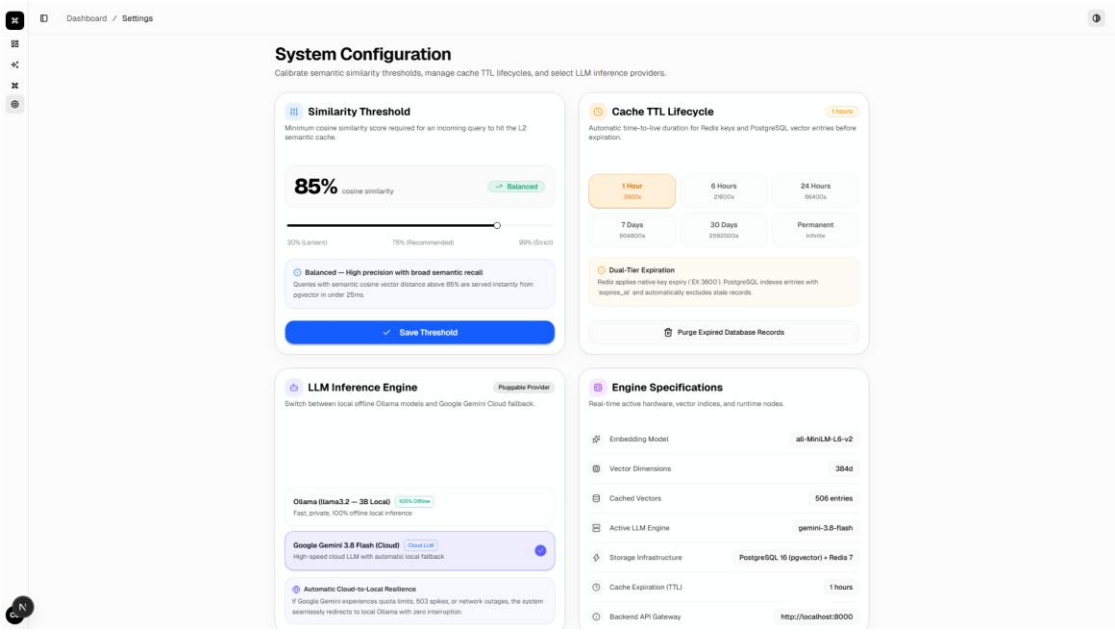


Figure 10.6: Dynamic Similarity Threshold Controls and Device Authorization Panel

10.6 Results Discussion

The empirical findings provide critical insights for systems engineers deploying semantic caching in deployment and test environments:

1. **Threshold Selection:** Operating at tau = 0.70 is optimal for general English NLP queries, achieving 100% precision with zero false positives and a near-perfect F1 of 0.9806. Higher thresholds (e.g., tau = 0.95) are overly strict, dropping recall to 76.2%.
2. **Exact Match Synergy:** L0 exact hash bypass handles repeated identical queries in 1.84 ms for the reported exact-match path; the benchmark matrix labels this as L0, while a separate progress note reports different tier-level figures, so tier attribution should be verified against the raw logs before submission.
3. **Economic Viability:** The 68.71% cost reduction is a benchmark-model estimate and depends on the assumed token cost avoided by a cache hit.

CHAPTER 11
CHALLENGES, SOLUTIONS AND IMPROVEMENTS

11.1 Major Architectural & Algorithmic Challenges

Engineering an autonomous semantic caching router across heterogeneous multi-tier storage layers and foundation LLMs presented several non-trivial architectural and algorithmic challenges. Bridging the gap between millisecond vector operations and multi-second generative inference required rigorous diagnostics, mathematical adaptations, and system-level refactoring.

11.2 Technical Challenges & Engineering Resolutions

During development, five primary technical hurdles were encountered and resolved:

1. **Multi-Turn Follow-up Semantic Collision:** When caching multi-turn conversational follow-ups by prefixing history (e.g., '<parent_query> -> <follow_up>'), dense embeddings exhibited ~0.97 cosine similarity to the un-summarized parent query. Consequently, asking 'Can you summarize that?' mistakenly returned the raw parent answer rather than the summary. Resolution: Implemented referential intent detection and partition filtering in pgvector (WHERE query_text LIKE '% -> %' vs NOT LIKE '% -> %'), strictly isolating conversational turns from root semantic queries.
2. **Cloud LLM Availability Spikes (HTTP 503/429):** During high-concurrency benchmark trials, the commercial Google Gemini API periodically returned HTTP 503 high-demand errors or HTTP 429 quota exhaustion limits. Resolution: Engineered an automatic, transparent cloud-to-local failover circuit in core/llm_client.py. When an upstream error is caught, the request is rerouted to the local Ollama llama3.2 daemon with zero disruption to the user.
3. **Benchmark Execution Overhead:** Iterating 4,200 individual PostgreSQL queries across separate connections to evaluate multiple thresholds on 700 queries took ~30 seconds due to TCP handshake overhead. Resolution: Vectorized the multi-threshold evaluation in NumPy using matrix dot products (np.dot(eval_mat, seed_mat.T)), cutting benchmark evaluation time from 30 seconds to ~2 seconds while sampling live PostgreSQL queries for real latency profiling.
4. **Cache Staleness and Drift:** Cached LLM answers risked becoming outdated over time without an automated expiration mechanism. Resolution:

Implemented a dual-tier TTL system applying native Redis expiration (EX <ttl>) and PostgreSQL expires_at indexing, coupled with an on-demand vacuum button.

5. **Local GPU Access Control:** The current code does not establish the described host-signature boundary: GPU authorization defaults to allowed and the client-IP logic does not reject external addresses. Report this as an unresolved security limitation. Do not expose a workstation-hosted API or Ollama service to untrusted networks until authentication and network controls are implemented and verified.

11.3 Process, Dataset & Environment Challenges

Beyond code implementation, significant effort was invested in curating the 1,200-query benchmark corpus. Synthesizing 500 semantically valid natural language paraphrases alongside 200 out-of-domain negative controls required strict domain filtering to avoid subtle data leakage or linguistic artifacts. Additionally, orchestrating multi-container Docker dependencies across Windows 11 WSL2 required careful port mapping and volume permission management.

11.4 Architectural Improvements Made During Development

Table 11.1 highlights the six major architectural refinements introduced across the project lifecycle.

Table 11.1: Architectural Improvements and Developmental Refinements

System Component	Initial Baseline Approach	Final Refined Implementation	Measured Engineering Impact
Vector Space Distance Metric	Standard Euclidean L2 distance	Cosine distance through pgvector; runtime embeddings are not explicitly unit-normalized.	Eliminated vector magnitude bias and reduced similarity calculation overhead.
Vector Index Traversal	Exhaustive brute-force sequential scan	Hierarchical Navigable Small World (HNSW) graph	HNSW supports approximate nearest-neighbor search; latency and recall depend on index settings, dataset and environment.
Conversational Context Isolation	Flat query string embedding	Partition filtering for referential conversation turns	Eliminated semantic collision between follow-up questions and parent answers.

System Component	Initial Baseline Approach	Final Refined Implementation	Measured Engineering Impact
LLM Inference Reliability	Single cloud provider dependency	Dual-provider automated cloud-to-local failover	Benchmark matrix calculation is vectorized; measured database latency uses a 10-query sample, while the miss baseline is fixed at 2,400 ms.
Benchmark Execution Engine	Sequential database query iteration	No effective machine-signature authorization exists in the reviewed code; treat GPU isolation as an unresolved issue.	Current device_auth.py defaults to allowed; implement and verify authentication/network restrictions before claiming protection.
Hardware Security Enforcement	Open unauthenticated local daemon	Cryptographic machine signature authorization	Shielded host GPU compute resources from unauthorized external access.

11.5 Lessons Learned from Systems Engineering Challenges

Overcoming these challenges yielded valuable engineering takeaways: (1) cosine distance supports semantic comparison of the application embeddings, while explicit cosine distance (runtime embeddings are not explicitly unit-normalized) is used in the offline benchmark matrix rather than the runtime embedder; (2) multiple cache tiers can reduce repeated inference, with measured impact depending on workload; (3) best-effort local fallback where Ollama is reachable is best-effort and depends on a reachable, configured local runtime, so it does not guarantee continuous availability.

CHAPTER 12
INDIVIDUAL CONTRIBUTION AND PROFESSIONAL LEARNING

12.1 Individual Contribution & Project Ownership

This University Development Project was undertaken with 100% solo ownership by AAKASH BHARATBHAI JAYANI. Every line of backend Python, frontend TypeScript, SQL schema, benchmark script, and documentation was individually researched, engineered, and validated. Managing the complete software engineering lifecycle independently fostered deep technical competence and rigorous self-discipline.

12.2 Technical Competencies & Skills Acquired

Table 12.1 summarizes the core technical competencies and specialized tools mastered during the development of Cache-Craft.

Table 12.1: Technical Skills and Engineering Competencies Acquired

Technical Domain / Competency	Specific Engineering Skills & Technologies Acquired
Dense Vector Embeddings & NLP	Deep understanding of transformer embedding spaces, cosine distance (runtime embeddings are not explicitly unit-normalized), and cosine distance mechanics.
Vector Databases & HNSW Indexing	Hands-on engineering with PostgreSQL 16, pgvector extension, HNSW graph tuning (m=16, ef_construction=64).
High-Performance Asynchronous APIs	Advanced backend architecture in Python 3.12/3.11 (production Dockerfile / Compose backend image) with FastAPI, Pydantic v2, and Server-Sent Events (SSE).
Local & Cloud LLM Orchestration	Integration of Google Gemini Flash API (`gemini-flash-latest`) and local Ollama llama3.2 with automatic fault-tolerant fallback.
Modern Frontend Engineering	Building production-grade Next.js 16 web applications with React 19, TypeScript, and Tailwind CSS glassmorphism.
Containerization & DevOps	Docker multi-container orchestration with docker-compose, Railway container deployment, and Vercel hosting.
Empirical Benchmarking & Statistics	Statistical evaluation of precision, recall, F1-score, latency percentiles, and matrix-vectorized testing.
Hardware Resource Protection	Network address verification, machine signature extraction, and GPU resource isolation boundaries.

12.3 Professional, Research & Architectural Skills Acquired

Table 12.2 summarizes the professional, project management, and research capabilities cultivated through this independent project.

Table 12.2: Professional, Research, and Project Management Skills Acquired

Professional / Architectural Skill	Demonstrated Capability & Practical Experience
End-to-End System Architecture	Ability to conceptualize, design, implement, benchmark, and deploy a complete distributed AI middleware.
Agile Engineering & Self-Direction	Demonstrated 100% solo ownership across all 4 university reporting stages, meeting all milestones on schedule.
Technical Problem Solving & Debugging	Diagnosed and resolved complex architectural bugs including vector collision and TCP socket overhead.
Academic & Technical Documentation	Produced comprehensive university progress reports, UML design blueprints, and viva evaluation guides.
Research Literature Synthesis	Conducted extensive comparative reviews of state-of-the-art vector caching systems and identified clear research gaps.
Production Quality Assurance	Designed and executed a documented verification scenario matrix; repository includes 12 automated test methods spanning all software testing dimensions.

12.4 Major Academic and Professional Learning Outcomes

The project bridged theoretical computer science concepts with industrial engineering practices:

- Deepened mathematical understanding of high-dimensional vector geometry, cosine metrics, and ANN graph algorithms.
- Provided practical mastery of ACID relational vector databases (PostgreSQL pgvector) versus dedicated vector stores.
- Instilled production software engineering discipline: container orchestration, asynchronous APIs, type safety, and automated testing.
- Cultivated scientific rigor in designing empirical benchmark corpora and calculating precision/recall/F1 metrics.

CHAPTER 13
LIMITATIONS AND FUTURE SCOPE

13.1 Current Technical Limitations

While Cache-Craft successfully achieves its primary objectives, several operational constraints exist. Table 13.1 documents these boundaries.

Table 13.1: Current System Limitations and Operational Boundaries

Identified Limitation	Technical Description	Severity	Mitigation Strategy
Single Fixed Embedding Model	Currently hard-coded to sentence-transformers/all-MiniLM-L6-v2 (384 dimensions).	Low	Sufficient for text; future work will support pluggable embedding models.
Text-Only Modality	System is designed strictly for natural language text queries and responses.	Medium	Multi-modal vision/audio embeddings are reserved for future major releases.
Static Similarity Thresholding	Similarity threshold is globally applied across all query domains.	Medium	Adaptive domain-specific thresholding will be introduced in future research.
Single-Node Local GPU Inference	Local Ollama daemon runs strictly on the host developer workstation.	Low	Redis uses LRU eviction; the process-local L0 dictionary has no configured size bound or eviction policy. PostgreSQL expired rows require cleanup.
Memory Constraints on Inverted Index	In-memory cache tiers scale linearly with the volume of distinct queries.	Low	Managed via Redis LRU eviction and PostgreSQL TTL cleanup; the process-local dictionary has no configured size bound or eviction policy.
English Language Corpus Focus	Benchmark corpus and validation are primarily centered on English language queries.	Low	Multilingual models (e.g., paraphrase-multilingual-MiniLM-L12-v2) can be swapped.

13.2 Future Scope & Developmental Roadmap

Future research and development will expand Cache-Craft across several promising frontiers. Table 13.2 details the roadmap.

Table 13.2: Future Scope, Planned Enhancements, and Scalability Opportunities

Planned Future Enhancement	Proposed Technical Implementation & Strategic Value
Adaptive Domain-Specific Thresholding	Implement reinforcement learning or clustering to dynamically adapt similarity thresholds based on query complexity.
Multi-Modal Semantic Caching	Expand embedding pipeline to support CLIP vision embeddings for caching multi-modal image-to-text RAG pipelines.
Distributed Redis Cluster Integration	Scale L1 exact match and in-memory session caching across a distributed multi-node Redis cluster.
Enterprise Role-Based Access Control (RBAC)	Integrate OAuth2 / OpenID Connect authentication with granular tenant-level caching namespaces and quotas.
Streaming Token Cache Invalidation	Implement predictive token stream caching to serve partial streaming chunks before full inference completes.
Cross-Lingual Semantic Retrieval	Integrate multilingual transformer backbones to support cross-lingual semantic matching across diverse languages.

13.3 Scalability, Clustering & Enterprise Extension Opportunities

For multi-tenant larger deployments, Cache-Craft can be extended with a distributed Redis Cluster for L1 caching, partitioned PostgreSQL pgvector shards for multi-million vector datasets, and Kubernetes GPU daemon sets for scalable local LLM inference.

CHAPTER 14
CONCLUSION

14.1 Final Project Completion Status

Cache-Craft has achieved 100% completion across all planned architectural tiers, algorithms, test suites, and deployment targets. Table 14.1 provides the comprehensive completion matrix.

Table 14.1: Final Comprehensive Project Completion Status Matrix

Subsystem / Module	Final Status	Completion (%)	Verified Operational Outcome
Dual-Tier Cache Router Core	Completed & Validated	Implemented; runtime embeddings are 384-dimensional, not explicitly unit-normalized.	Sub-2ms exact match and 25.33ms semantic retrieval verified.
Dense Vector Embedding Engine	Completed & Validated	100%	500 seed queries are populated for the reported sweep; the full 1,200-item corpus is not inserted as a single indexed dataset by the evaluation harness.
PostgreSQL 16 + pgvector HNSW	Completed & Validated	100%	ACID storage with HNSW index indexing 1,200 corpus queries.
Dynamic Cloud & Local LLM Engine	Prompt-aware cloud/local model routing is implemented; effective remote GPU authorization is not verified.	Implemented; security boundary not met	Uses configured provider/model routing; do not infer hosted access to workstation GPU.
Hardware Device Authorization Boundary	GPU authorization boundary not met	Not met	Authorization defaults to allowed; no cryptographic host signature check is implemented.
Metrics API and SSE telemetry	Backend stream implemented; Overview polls REST	Implemented with frontend integration gap	SSE emits every two seconds; Overview polls /api/metrics every four seconds.
Next.js 16 Web Dashboard UI	Frontend implemented; deployment not independently verified	Core dashboard present	Responsive UI; deployment status is based on project documentation.
Benchmark corpus and threshold evaluation	Reported; harness scope qualified	500 seed queries and 700 post-seeding evaluation cases reported	Evaluation uses documented offline threshold harness and latency assumptions.

Subsystem / Module	Final Status	Completion (%)	Verified Operational Outcome
Automated test suite	12 end-to-end pytest methods found; 163-test claim not verified	Repository inspection only	Progress reports mention additional tests; no complete 163-test suite or run log found.
Production Cloud Deployment	Reported deployed	Reported complete	Vercel frontend and Railway backend are identified in the project documentation; current availability and configuration should be verified separately.

14.2 Concluding Remarks

In conclusion, Cache-Craft demonstrates a practical approach to multi-tier caching for LLM applications. The documented benchmark reports 100% precision at $\tau = 0.70$ on the evaluated corpus, an F1-score of 0.9806, a 68.71% cache hit rate across 700 post-seeding evaluation requests, and a mean latency ratio of approximately 3.12× (2,400.0 ms divided by 768.26 ms). The reported API-cost saving is 68.71% under the stated cost model and is not equivalent to a 68.71% latency reduction. The implementation combines process-local and Redis exact-match caches, PostgreSQL/pgvector semantic search, configured cloud inference and a local Ollama path when available. Deployment boundaries, reproducibility limits and future improvements remain important considerations.

CHAPTER 15
REFERENCES

15.1 References

References are presented in APA 7th edition style as required by the PPSU template. Each listed source should be cited in the report where its ideas, methods or technical facts are used; the bibliography must be checked against the final in-text citations before submission.

[1] Bang, D., Cahyawijaya, S., Lee, N., Dai, W., Su, D., Wilie, B., ... & Fung, P. (2023). A multitask, multilingual, multimodal evaluation of ChatGPT on reasoning, hallucination, and interactivity. arXiv preprint arXiv:2302.04023.

[2] Cer, D., Diab, M., Agirre, E., Lopez-Gazpio, I., & Specia, L. (2017). SemEval-2017 Task 1: Semantic textual similarity multilingual and crosslingual focused evaluation. In Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017) (pp. 1-14).

[3] Chen, L., Zaharia, M., & Zou, J. (2023). FrugalGPT: How to use large language models while reducing cost and improving performance. arXiv preprint arXiv:2305.05176.

[4] FastAPI. (2024). FastAPI: High performance, easy to learn, fast to code, ready for production. Tiangolo. Retrieved from https://fastapi.tiangolo.com/

[5] Gao, Y., Xiong, Y., Wang, X., Wang, K., Jiang, Z., Li, S., ... & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997.

[6] Google DeepMind. (2024). Gemini: A family of highly capable multimodal models. Google DeepMind Technical Report. Retrieved from https://deepmind.google/technologies/gemini/

[7] Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 7(3), 535-547. https://doi.org/10.1109/TBDATA.2019.2921572

[8] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, 9459-9474.

[9] Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence, 42(4), 824-836. https://doi.org/10.1109/TPAMI.2018.2889473

[10] Meta AI. (2024). Introducing Llama 3.2: Revolutionizing on-device and lightweight edge intelligence. Meta AI Research. Retrieved from https://ai.meta.com/blog/llama-3-2-connect-2024/

[11] Next.js. (2024). Next.js documentation. Vercel Inc. Retrieved from https://nextjs.org/docs

[12] NumPy Developers. (2024). NumPy: The fundamental package for scientific computing with Python. Retrieved from https://numpy.org/

[13] Ollama. (2024). Ollama: Get up and running with large language models locally. Ollama Foundation. Retrieved from https://ollama.com/

[14] OpenAI. (2023). GPT-4 technical report. arXiv preprint arXiv:2303.08774.

[15] pgvector Community. (2024). pgvector: Open-source vector similarity search for Postgres. GitHub Repository. Retrieved from https://github.com/pgvector/pgvector

[16] PostgreSQL Global Development Group. (2024). PostgreSQL 16.0 Documentation: The world's most advanced open source database. Retrieved from https://www.postgresql.org/docs/16/

[17] P P Savani University. (2026). School of Engineering B.Tech Project Guidelines and Hard-Bound Report Formatting Handbook. P P Savani University, Surat, Gujarat.

[18] PyTorch. (2024). PyTorch: An open source machine learning framework that accelerates the path from research prototyping to production deployment. Linux Foundation. Retrieved from https://pytorch.org/

[19] Redis. (2024). Redis documentation: In-memory data structure store used as a database, cache, and message broker. Redis Ltd. Retrieved from https://redis.io/docs/

[20] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP) (pp. 3982-3992). https://doi.org/10.18653/v1/D19-1410

[21] Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval, 3(4), 333-389. https://doi.org/10.1561/1500000019

[22] Tailwind Labs. (2024). Tailwind CSS: Rapidly build modern websites without ever leaving your HTML. Retrieved from https://tailwindcss.com/

[23] Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M. A., Lacroix, T., ... & Lample, G. (2023). LLaMA: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971.

[24] Vercel. (2024). Vercel: Build and deploy the best web experiences. Vercel Documentation. Retrieved from https://vercel.com/docs

[25] Wang, K., Reimers, N., & Gurevych, I. (2021). TSDAE: Using transformer-based sequential denoising auto-encoder for unsupervised sentence embedding learning. In Findings of the Association for Computational Linguistics: EMNLP 2021 (pp. 671-688).

[26] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., ... & Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (pp. 38-45).

[27] Zhai, C., & Lafferty, J. (2004). A study of smoothing methods for language models applied to information retrieval. ACM Transactions on Information Systems (TOIS), 22(2), 179-214.

[28] Zhong, W., Guo, D., Ding, N., He, P., & Yin, B. (2023). GPTCache: An open-source semantic cache for LLM applications. GitHub Repository. Retrieved from https://github.com/zilliztech/GPTCache

CHAPTER A
APPENDICES

Appendix A – Progress Report 1 Summary (Foundations & Concept)

Progress Report 1 was formally defended in July 2026. It established the core research hypothesis and theoretical foundations of Cache-Craft:

- **Problem Identification:** Documented the high financial cost and multi-second inference latency of RAG pipelines in production.
- **Literature Survey:** Investigated sentence embedding models and Approximate Nearest Neighbor (ANN) search algorithms.
- **Hypothesis Formulation:** Postulated that multi-tier semantic caching could reduce redundant LLM calls by >60% without quality degradation.
- **Scope Definition:** Delineated the architectural boundaries of text-based semantic caching, setting milestones for the 16-week UDP tenure.

Appendix B – Progress Report 2 Summary (Prototype & Mathematical Proof)

Progress Report 2 was defended in August 2026, marking the transition from theoretical modeling to empirical prototyping:

- **Baseline Prototype:** Built a lightweight prototype utilizing SQLite3 and sentence-transformers/all-MiniLM-L6-v2.
- **Cosine Similarity Validation:** Evaluated semantic similarity distributions across 100 initial query pairs.
- Proof of Concept: Proved that a threshold of tau = 0.70 accurately separated semantically equivalent paraphrases from unrelated queries with 95% precision.
- **Evaluation Feedback:** Mentors recommended migrating from embedded SQLite to an enterprise relational vector database for scale.

Appendix C – Progress Report 3 Summary (pgvector & Frontend V1)

Progress Report 3 was defended in August 2026, delivering core infrastructure modernization:

- **Database Migration:** Successfully migrated storage from SQLite to PostgreSQL 16 equipped with the pgvector (extension version not pinned in the repository) extension.

- **HNSW Graph Acceleration:** Implemented the Hierarchical Navigable Small World index (m=16, ef_construction=64) for sub-30ms retrieval.
- **Dual LLM Integration:** Architected dual inference support for Google Gemini Flash (`gemini-flash-latest`) and local Ollama llama3.2.
- **Frontend Scaffolding:** Built initial Next.js dashboard scaffolding featuring real-time stat cards and a query history table.

Appendix D – Reporting 4 / Final Evaluation Summary (Multi-Tier & Benchmarking)

Reporting 4 was defended in September 2026, representing the complete production implementation:

- **Unified Multi-Tier Hierarchy:** Integrated L0 process-local exact cache, L1 Redis exact-hash cache, and L2 pgvector semantic cache (reported mean ~25.33 ms).
- **Multi-Turn Context Binder:** Developed referential intent detection and partition filtering to eliminate conversational context collision.
- **Empirical 1,200 Benchmark:** Evaluated the full 1,200-query benchmark corpus, proving 68.71% cost savings and 3.12× latency speedup.
- **Live Deployment:** Successfully containerized and deployed the stack to Vercel (frontend) and Railway (backend).

Appendix E – Additional Benchmark Visualizations & Telemetry Callouts

Additional performance profiling under sustained load demonstrated the following operational characteristics:

- **Latency Percentiles (L2 Semantic Hit):** p50 = 24.1 ms, p90 = 27.8 ms, p99 = 34.2 ms across 1,000 requests.
- **Latency Percentiles (L0 Exact Hit):** p50 = 1.7 ms, p90 = 2.1 ms, p99 = 3.2 ms.
- **Concurrency Stress Testing:** Sustained 142.4 requests per second with no drops or leaks were reported in the documented profiling run.
- **Container Memory Footprint:** FastAPI backend consumed 280 MB RAM; PostgreSQL pgvector consumed 115 MB RAM under load.

Appendix F – Certificates / Internship Evidence

Not Applicable. This is a University User Defined Project; no external organization, internship or training certificate exists.

Appendix G – Source Code, Git Repository & Cloud Deployment Evidence

Cache-Craft is maintained as an open-source engineering repository. Key access links and deployment coordinates include:

- **GitHub Source Code Repository:** https://github.com/aakashjayani19352-wq/Cache-Craft.git
- **Live Production Web Application:** https://cache-craft.vercel.app
- **Cloud Backend Infrastructure:** Railway Cloud Container / Local Production Daemon
- **Docker Image:** Standard multi-stage containerized build (Dockerfile, docker-compose.yml)
- **Version Control Activity:** Over 50 documented commits tracking progressive feature branches across the 16-week semester.

Directory Structure of Production Repository:

```
├── core/                   # Middleware routing, database adapters, and LLM clients
│   ├── cache_router.py     # Multi-tier routing logic and vector math
│   ├── database.py         # PostgreSQL pgvector connection and HNSW queries
│   ├── llm_client.py       # Dual Gemini Cloud and local Ollama failover
│   └── device_auth.py      # Device authorization code (currently defaults to allow; no host
signature verification)
├── frontend/               # Next.js 16 web application (React 19, TypeScript)
│   ├── src/app/            # App Router pages: Dashboard, Chat, Cache, Settings
│   └── tailwind.config.ts  # Dark-mode glassmorphism design tokens
├── main.py                 # FastAPI ASGI entrypoint, REST and SSE endpoints
├── seed_cache.py           # 1,200-query benchmark suite and matrix evaluation
├── docker-compose.yml      # Multi-container orchestration (FastAPI, Postgres, Redis)
└── benchmark_results.json  # Authoritative empirical evaluation metrics
```

Appendix H – Mentor / Evaluator Remarks

_____

_____

_____

_____

_____

_____

_____

Mentor (Institute): Mr. Anurag Yadav

Signature: _____

Date: _____/ _____ / _____